

Estacionamiento Inteligente con Control de Acceso Automatizado

Nicolle Rosatti
Joaquín Tissera

Profesor: Ariel Gonzalez
Materia: Simulación (Cód. 1962)
Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas Físico-Químicas y Naturales
Licenciatura en Ciencias de la Computación
2025

Resumen

El presente trabajo aborda el modelado y simulación de un sistema automatizado de control de acceso vehicular a un estacionamiento con capacidad limitada. El objetivo principal es garantizar un funcionamiento eficiente y seguro del sistema, evitando el ingreso de vehículos cuando se alcanza la capacidad máxima y asegurando que todo vehículo que ingrese pueda salir sin conflictos operativos.

Para ello, se implementa una lógica de control basada en sensores, barreras automáticas y un controlador central, respetando propiedades fundamentales como *Safety* (seguridad) y *Liveness* (vivacidad). Además, se analizan métricas clave de rendimiento tales como el tiempo medio de permanencia de los vehículos, la tasa de rechazo por falta de espacio, y el porcentaje de ocupación promedio del estacionamiento.

Índice

1. Introducción	3
1.1. Contexto y motivación	3
1.2. Objetivos	3
2. Descripción del sistema	3
2.1. Descripción Funcional	3
2.2. Formalismo DEVS y CML-DEVS	5
2.3. Propiedades de interés	5
2.4. Requisitos Técnicos	5

2.5.	Componentes Modelados	6
2.6.	Resumen de parámetros temporales	6
3.	Especificación CML-DEVS de los Modelos Atómicos	7
3.1.	Diagrama de caja de cada modelo atómico (inputs/outputs)	7
3.2.	Especificación CML-DEVS (Conceptual Modeling Language - Discrete Event System Specification)	11
3.2.1.	GeneradorArribos	11
3.2.2.	SensorEntrada	11
3.2.3.	SensorSalida	13
3.2.4.	BarreraEntrada	15
3.2.5.	BarreraSalida	17
3.2.6.	Controlador	18
3.2.7.	Estacionamiento	20
3.2.8.	Monitor	22
4.	Implementación en PowerDEVS	25
4.1.	Componentes parametrizables	25
4.2.	Conexión de modelos	27
4.3.	Configuración de la simulación	27
4.4.	Tabla de parámetros principales	28
5.	Diseño Experimental y Métricas	28
5.1.	Métricas a capturar	28
6.	Casos propuestos para las corridas:	29
7.	Experimentos, Resultados y Gráficos	30
7.1.	Experimento 1	30
7.2.	Experimento 2	31
7.3.	Experimento 3	33
7.4.	Experimento 4	34
7.5.	Experimento 5	36
7.6.	Experimento 6	37
7.7.	Experimento 7	39
7.8.	Experimento 8	40
7.9.	Experimento 9	42
7.10.	Experimento 10	43
8.	Intervalo de confianza	46
8.1.	Intervalo de confianza del 95 % para la Tasa de Rechazo	46
8.2.	Intervalo de confianza del 95 % para el Tiempo de Permanencia	46
8.3.	Intervalo de confianza del 95 % para la Ocupación Promedio	47
8.4.	Visualización de los intervalos de confianza	47
8.5.	Conclusión	48

9. Análisis de Propiedades	49
9.1. Safety	49
9.2. Liveness 1	49
9.3. Liveness 2	49
10. Discusión y Sugerencias de Mejora	50
10.1. Impacto de parámetros en rechazos y ocupación.	50
10.2. Posible extensión: múltiples barreras, mayores velocidades o capacidad en el estacionamiento.	50
11. Conclusiones	51
11.1. Recomendaciones para la operación real	51
11.2. Líneas futuras de investigación	51
11.3. Reflexión final	52
12. Referencias	52

1. Introducción

1.1. Contexto y motivación

Necesidad de simular sistemas de acceso a estacionamientos automatizados.

1.2. Objetivos

- Modelar el sistema en CML-DEVS.
- Verificar propiedades de *safety* y *liveness*.
- Medir métricas clave: ocupación, rechazos, tiempo de estancia.

2. Descripción del sistema

2.1. Descripción Funcional

El sistema simula el comportamiento de un estacionamiento automatizado con acceso controlado. A continuación se describe el flujo típico de un vehículo:

1. **Llegada al sistema:** los vehículos arriban de forma estocástica. El tiempo entre llegadas sigue una distribución exponencial con media de 10 segundos, simulando condiciones de tráfico variable.

2. Detección en el sensor de entrada:

- a) Si no hay vehículos en espera, el vehículo es detectado tras una latencia fija de 1 segundo.

- b) Si hay vehículos esperando ingresar, el nuevo vehículo se encola en la fila de espera del sensor.

El sensor notificará al controlador cuando se haya terminado de sensar un vehículo.

3. **Decisión de ingreso:** el controlador evalúa si la ocupación del estacionamiento permite un nuevo ingreso. Puede:

- **Admitir:**

- **AdmitidosDirecto:** se autoriza el ingreso del vehículo inmediatamente.
- **AdmitidosConDemora:** el vehículo permanece en espera durante un máximo de 3 segundos, aguardando que se libere un lugar en el estacionamiento. Si se libera una plaza en ese lapso, se le permite ingresar; de lo contrario, es rechazado.

- **Rechazar:** el vehículo se retira de la barrera y del sistema en un tiempo de 2 segundos.

4. **Proceso de ingreso (si fue aceptado):**

- a) La barrera de entrada tarda 4 segundos en abrirse.
- b) El vehículo cruza en un tiempo aleatorio (siguiendo una distribución uniforme) entre 1 y 3 segundos.
- c) La barrera tarda 4 segundos en cerrarse.
- d) Se registra el evento `vehiculoHaIngresado` al finalizar el cruce, se notifica al controlador.

5. **Permanencia en el estacionamiento:** el vehículo permanece dentro durante un tiempo aleatorio entre 120 y 300 segundos (con una distribución uniforme), simulando su estadía.

6. **Salida del sistema:** Una vez cumplido el tiempo de permanencia dentro del estacionamiento, el auto desea retirarse.

- a) El vehículo llega al sensor de salida, que lo detecta con una latencia fija de 1 segundo.
- b) Si la barrera aún no está cerrada por completo o hay otro vehículo saliendo, se encola y espera.
- c) El sensor de salida notifica al controlador cuando hay un vehículo esperando egresar.
- d) A diferencia de la entrada, si no hay obstrucción, la salida es autorizada inmediatamente por el controlador.

7. **Proceso de egreso:**

- a) La barrera de salida tarda 4 segundos en abrirse.

- b) El vehículo cruza en un tiempo aleatorio uniforme entre 1 y 3 segundos.
 - c) La barrera tarda 4 segundos en cerrarse.
 - d) Se registra el evento `vehiculoHaSalido`.
8. **Restricción importante:** solo un vehículo puede cruzar la barrera de entrada/salida a la vez (y existe una de entrada y otra distinta de salida, son independientes). Las solicitudes concurrentes son manejadas secuencialmente mediante una cola.

2.2. Formalismo DEVS y CML-DEVS

El sistema fue modelado utilizando el formalismo DEVS (Discrete Event System Specification) y su variante textual CML-DEVS. Se diseñaron modelos atómicos para representar los componentes individuales del sistema y un modelo acoplado para definir la interacción entre ellos. Cada modelo atómico contiene sus propios puertos de entrada y salida, variables de estado y funciones de transición interna, externa y de salida.

2.3. Propiedades de interés

El sistema debe cumplir con dos propiedades fundamentales:

- **Safety:** la ocupación del estacionamiento nunca debe superar la capacidad máxima definida ($K = 30$). Esta propiedad garantiza que no ingresen más vehículos de los que pueden ser alojados.
- **Liveness 1:** todo vehículo admitido debe eventualmente egresar. Esto implica que no deben quedar autos eternamente bloqueados dentro del estacionamiento y permite verificar que el flujo de entrada y salida se mantenga operativo.
- **Liveness 2:** todo vehículo que ha ingresado no puede estar más del 1 % del tiempo que estuvo estacionado para salir de la zona de estacionamiento. Es decir, si un vehículo estuvo 60 minutos estacionado, no puede tardar más de 0,6 minutos en salir. Esta demora puede deberse, por ejemplo, a la cola de salida.

2.4. Requisitos Técnicos

- **Capacidad máxima del estacionamiento:** $K = 30$ vehículos.
- **Tiempos de procesamiento:** sensores con latencia de 1 segundo; barreras con apertura y cierre de 4 segundos cada uno; cruce de vehículos modelado como una variable aleatoria uniforme entre 1 y 3 segundos.
- **Tiempo de permanencia:** modelado mediante una distribución uniforme entre 120 y 300 segundos.
- **Supuestos operativos:**
 - Los vehículos llegan de forma independiente (tiempo entre arribos exponencial).

- Los vehículos no retroceden ni cancelan su intención de ingreso.
- No existen fallas en sensores o barreras.

2.5. Componentes Modelados

Componente	Tipo DEVS	Función
Generador Aleatorio	Atómico	Genera los arribos de los autos con una distribución exponencial con media 10.
Sensor de Entrada	Atómico	Detecta la presencia de vehículos y activa el controlador.
Barrera de Entrada	Atómico	Controla físicamente el ingreso de vehículos al estacionamiento.
Controlador	Atómico	Coordina decisiones de admisión basadas en la ocupación actual y autoriza el egreso inmediato del estacionamiento.
Estacionamiento	Atómico	Gestiona el tiempo de permanencia
Sensor de Salida	Atómico	Detecta los vehículos que desean salir.
Barrera de Salida	Atómico	Permite el egreso de vehículos una vez autorizados.
Monitor	Atómico	Contiene y realiza todas las estadísticas necesarias del sistema.

Cuadro 1: Componentes modelados del sistema de estacionamiento inteligente.

2.6. Resumen de parámetros temporales

Parámetro	Valor sugerido
Capacidad máxima (N)	30 vehículos
Llegadas (interarribos)	Exponencial con media 10 s
Sensor de entrada	1 s de latencia
Decisión del controlador	Hasta 3 s
Apertura de barrera	4 s
Cruce del vehículo	Uniforme entre 1 y 3 s
Cierre de barrera	4 s
Tiempo de retiro	2 s (si no se permite el ingreso)
Permanencia	Uniforme entre 120 y 300 s
Sensor de salida	1 s de latencia

Cuadro 2: Resumen de parámetros temporales utilizados en la simulación

3. Especificación CML-DEVS de los Modelos Atómicos

3.1. Diagrama de caja de cada modelo atómico (inputs/outputs)

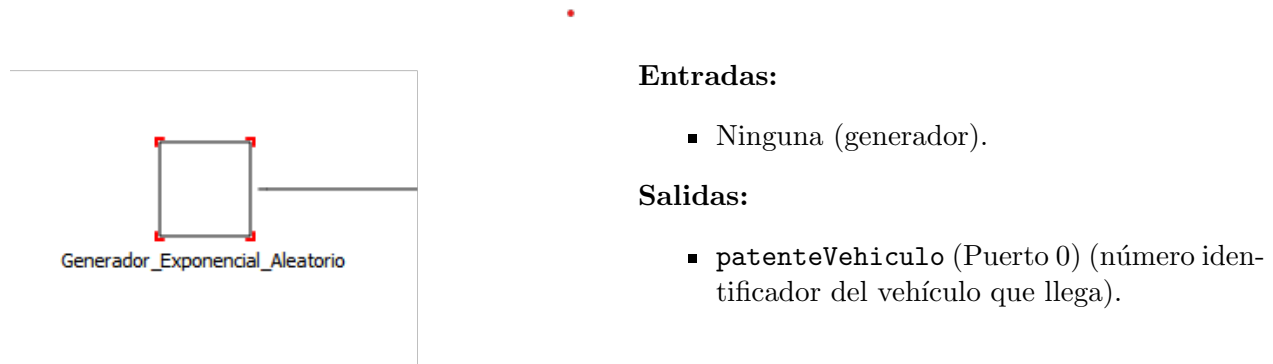


Figura 1: Dev del generador de arribos de los autos

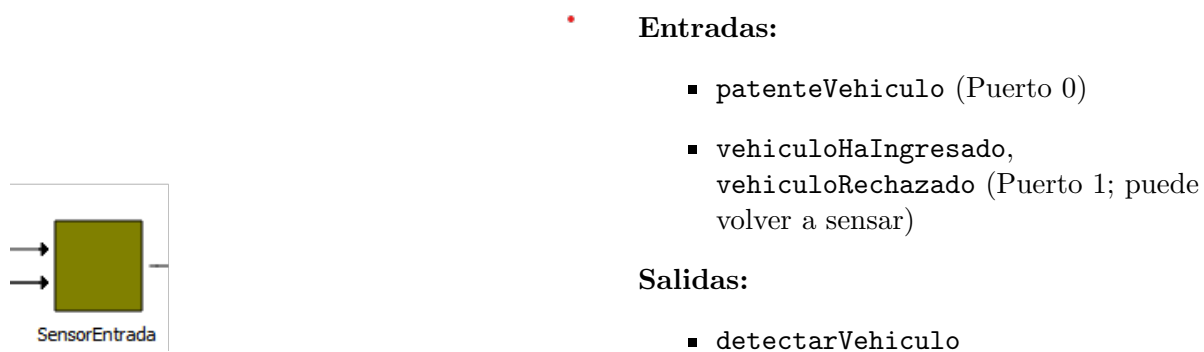


Figura 2: Componente SensorEntrada

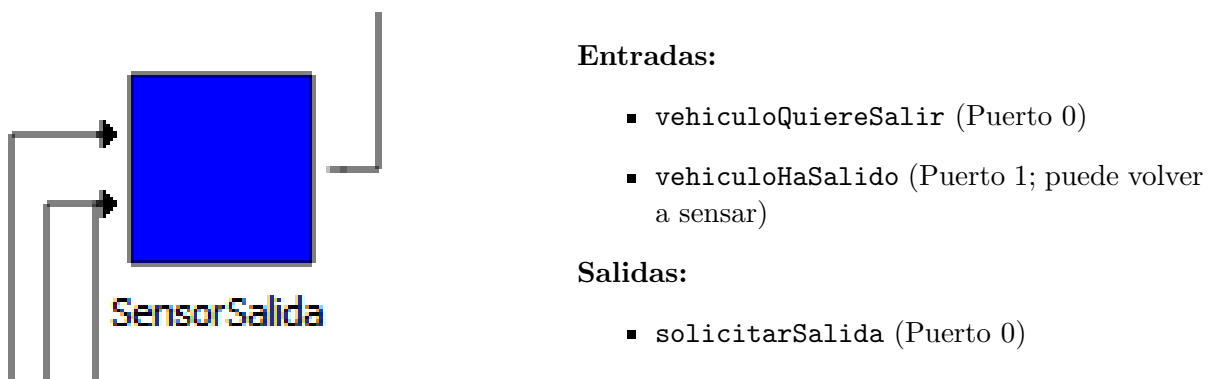


Figura 3: Componente SensorSalida

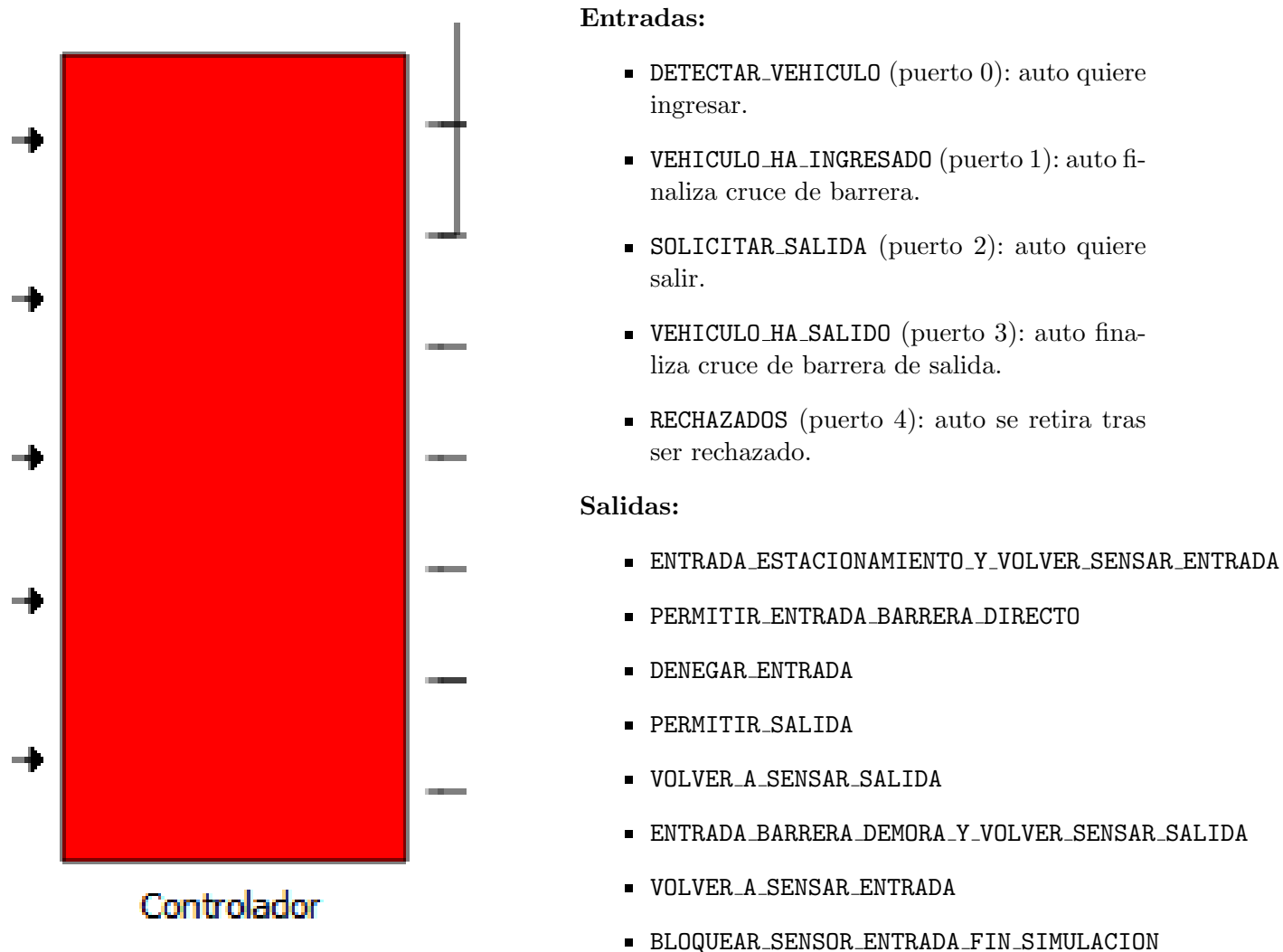
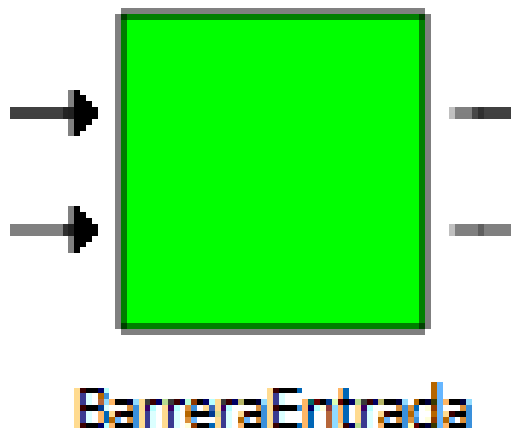


Figura 4: Componente Controlador: gestiona la lógica de ingreso y egreso según el estado del sistema.



Entradas:

- PERMITIR_ENTRADA (Puerto 0): un auto debe ingresar.
- DENEGAR_ENTRADA (Puerto 1): un auto se debe retirar de la barrera.

Salidas:

- VEHICULO_HA_INGRESADO (Puerto 0): un auto ingresado completó su paso por la barrera.
- VEHICULO_RECHAZADO_SE_HA_RETIRADO (Puerto 1): un auto rechazado completó su paso por la barrera.

Figura 5: Componente BarreraEntrada: aplica una latencia para autos que ingresan o son rechazados previo a estacionarse.



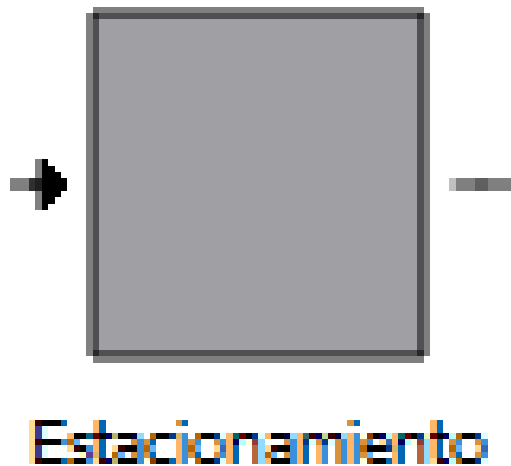
Entradas:

- PERMITIR_SALIDA (Puerto 0): un auto debe salir.

Salidas:

- VEHICULO_HA_SALIDO (Puerto 0): un auto que sale completó su paso por la barrera.

Figura 6: Componente BarreraSalida: aplica una latencia para autos que salen del estacionamiento.



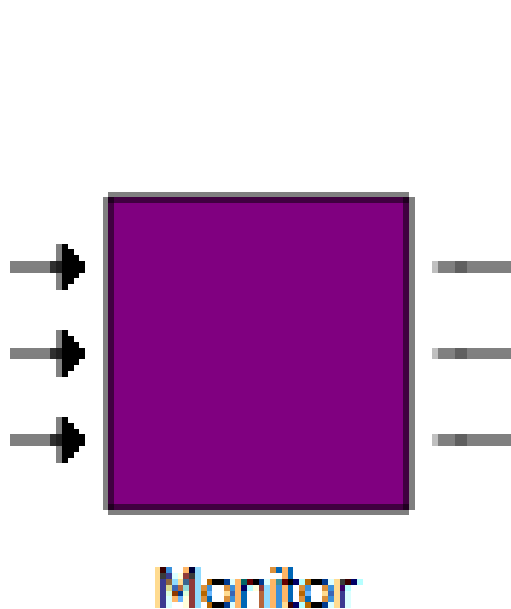
Entradas:

- PERMITIR_ENTRADA (Puerto 0): un auto ingresa al estacionamiento.

Salidas:

- VEHICULO QUIERE SALIR (Puerto 0): un auto completó su duración de permanencia en el estacionamiento.

Figura 7: Componente **Estacionamiento**: define una permanencia para autos se estacionan, siguiendo una distribución uniforme parametrizada.



Entradas:

- BARRERA_ENTRADA_ADMITIDOS (Puerto 0): los autos admitidos para estacionarse.
- BARRERA_ENTRADA_RECHAZADOS (Puerto 1): los autos rechazados para estacionarse.
- BARRERA_SALIDA_EGRESO_ABSOLUTO (Puerto 2): los autos que se retiraron definitivamente del sistema.

Salidas:

- TIEMPO_PROMEDIO_PERMANENCIA (Puerto 0): el estadístico ya calculado de tiempo promedio de permanencia.
- TASA_DE_RECHAZO (Puerto 1): el estadístico ya calculado de tasa de rechazo.
- OCUPACION_PROMEDIO (Puerto 2): el estadístico ya calculado de ocupación promedio.

Figura 8: Componente **Monitor**: calcula de manera pasiva los estadísticos de tiempo promedio permanencia, tasa de rechazo y ocupación promedio. Sus salidas se almacenan en disco y son dadas a GNUPlot para graficarlas como corresponde.

3.2. Especificación CML-DEVS (Conceptual Modeling Language - Discrete Event System Specification)

3.2.1. GeneradorArribos

```
atomic generador_arribos <S,  $S_0$ , Y,  $\delta_{int}$ ,  $\lambda$ , ta> is

  S is
     $\sigma$  :  $\mathbb{R}$ ;
    pat :  $\mathbb{N}$ ;
  end S

   $S_0$  (pat,  $\sigma$ )
    pat = 1;
     $\sigma$  =  $\infty$ ;
  end  $S_0$ 

  Y is
    patenteVehiculo_0 :  $\mathbb{N}$ ;
  end Y

   $\delta_{int}$ (pat,  $\sigma$ ) is
    pat = pat + 1;
     $\sigma$  = Exponencial(seed = 3, mean = 1/10);
  end  $\delta_{int}$ 

   $\lambda$ (pat,  $\sigma$ ) is
    patenteVehiculo_0 = pat;
  end  $\lambda$ 

  ta(pat,  $\sigma$ ) is
     $\sigma$ 
  end ta

end atomic
```

3.2.2. SensorEntrada

```
atomic sensor_entrada <X, S,  $S_0$ , Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

  X is
    patenteVehiculo_0 :  $\mathbb{N}$ ;
    volverASensar_1 :  $\mathbb{N}$ ;
  end X

  S is
```

```

     $\sigma$  :  $\mathbb{R}$ ;
    pat :  $\mathbb{N}$ ;
    bloqueado : Boolean;
    procesando : Boolean;
    fin_simulacion : Boolean;
    TIEMPO_MAX_ENTRADA_VEHICULOS:  $\mathbb{R}$ ;
    cola : Queue  $\mathbb{N}$ ;
    latencia :  $\mathbb{R}$ ;
end S

S0(pat,  $\sigma$  , bloqueado, procesando, fin_simulacion,
TIEMPO_MAX_ENTRADA_VEHICULOS, cola, latencia) is
    pat = 0;
    bloqueado = false;
    procesando = false;
    fin_simulacion = false;
    TIEMPO_MAX_ENTRADA_VEHICULOS = 43200
     $\sigma$  =  $\infty$ ;
    latencia = 1
end S0

Y is
    detectarVehiculo_0 :  $\mathbb{N}$ ;
end Y

 $\delta_{int}$ (pat,  $\sigma$ , bloqueado, procesando, fin_simulacion,
TIEMPO_MAX_ENTRADA_VEHICULOS, cola) is
    if tsim  $\geq$  TIEMPO_MAX_ENTRADA_VEHICULOS  $\rightarrow$ 
        bloqueado = true;
        fin_simulacion = true;
         $\sigma$  =  $\infty$ ;
    if tsim < TIEMPO_MAX_ENTRADA_VEHICULOS  $\rightarrow$ 
         $\sigma$  =  $\infty$ ;
        procesando = false;
    end  $\delta_{int}$ 

 $\delta_{ext}$ (pat,  $\sigma$  , bloqueado, procesando, fin_simulacion,
TIEMPO_MAX_ENTRADA_VEHICULOS, cola, latencia, e, x, port) is
    if port == 0  $\rightarrow$ 
        cola.push(x);
        if not fin_simulacion  $\rightarrow$ 
            if not bloqueado  $\rightarrow$ 
                procesando = true;
                pat = cola.front();
                cola.pop();
            end
        end
    end

```

```

        bloqueado = true;
         $\sigma$  = latencia;
    if bloqueado  $\rightarrow$ 
        if procesando  $\rightarrow$ 
             $\sigma$  =  $\sigma$  - e;
        if not procesando  $\rightarrow$ 
             $\sigma$  =  $\infty$ ;
    if fin_simulacion  $\rightarrow$ 
         $\sigma$  =  $\infty$ ;
if port == 1  $\rightarrow$ 
    if not fin_simulacion  $\rightarrow$ 
        bloqueado = false;
        if cola.empty()  $\rightarrow$ 
             $\sigma$  =  $\infty$ ;
        if not cola.empty()  $\rightarrow$ 
            pat = cola.front();
            cola.pop();
            procesando = true;
            bloqueado = true;
             $\sigma$  = latencia;
        if fin_simulacion  $\rightarrow$ 
             $\sigma$  =  $\infty$ ;
end  $\delta_{ext}$ 

     $\lambda$ (pat,  $\sigma$  , bloqueado, procesando, fin_simulacion,
TIEMPO_MAX_ENTRADA_VEHICULOS, cola, latencia) is
        detectarVehiculo_0: pat;
    end  $\lambda$ 

    ta(pat,  $\sigma$  , bloqueado, procesando, fin_simulacion,
TIEMPO_MAX_ENTRADA_VEHICULOS, cola, latencia) is
         $\sigma$ 
    end ta

end atomic

```

3.2.3. SensorSalida

```

atomic sensor_salida <X, S,  $S_0$ , Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

    X is
        vehiculoQuiereSalir_0 :  $\mathbb{N}$ ;
        volverASensar_1 :  $\mathbb{N}$ ;
    end X

```

```

S is
   $\sigma$  :  $\mathbb{R}$ ;
  pat :  $\mathbb{N}$ ;
  bloqueado : Boolean;
  procesando : Boolean;
  cola : Queue  $\mathbb{N}$ ;
  latencia :  $\mathbb{R}$ ;
end S

 $S_0$  (pat,  $\sigma$  , bloqueado, procesando, cola, latencia) is
  pat = 0;
  bloqueado = false;
  procesando = false;
   $\sigma$  =  $\infty$ ;
  latencia = 1
end  $S_0$ 

Y is
  solicitarSalida_0 :  $\mathbb{N}$ ;
end Y

 $\delta_{int}$ (pat,  $\sigma$  , bloqueado, procesando, cola, latencia) is
  procesando = false;
   $\sigma$  =  $\infty$ ;
end  $\delta_{int}$ 

 $\delta_{ext}$ (pat,  $\sigma$  , bloqueado, procesando, cola, latencia, e, x, port) is
  if port == 0  $\rightarrow$ 
    cola.push(x);
    if not bloqueado  $\rightarrow$ 
      procesando = true;
      pat = cola.front();
      cola.pop();
      bloqueado = true;
       $\sigma$  = latencia;
    if bloqueado  $\rightarrow$ 
      if procesando  $\rightarrow$ 
         $\sigma$  =  $\sigma$  - e;
      if not procesando  $\rightarrow$ 
         $\sigma$  =  $\infty$ ;
    if port == 1  $\rightarrow$ 
      bloqueado = false;
      if cola.empty()  $\rightarrow$ 
         $\sigma$  =  $\infty$ ;

```

```

        if not cola.empty() →
            pat = cola.front();
            cola.pop();
            procesando = true;
            bloqueado = true;
             $\sigma$  = latencia;
        end  $\delta_{ext}$ 

 $\lambda$ (pat,  $\sigma$ , bloqueado, procesando, cola, latencia) is
    solicitarSalida_0 : pat;
end  $\lambda$ 

ta(pat,  $\sigma$ , bloqueado, procesando, cola, latencia) is
     $\sigma$ 
end ta

end atomic

```

3.2.4. BarreraEntrada

```

atomic barrera_entrada <X, S,  $S_0$ , Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

X is
    permitirEntrada_0 :  $\mathbb{N}$ ;
    denegarEntrada_1 :  $\mathbb{N}$ ;
end X

S is
     $\sigma$  :  $\mathbb{R}$ ;
    pat :  $\mathbb{N}$ ;
    rechazado : Boolean;
    tiempo_abertura :  $\mathbb{R}$ ;
    tiempo_cierre :  $\mathbb{R}$ ;
    tiempo_retiro :  $\mathbb{R}$ ;
end S

 $S_0$  (pat,  $\sigma$ , rechazado, tiempo_abertura, tiempo_cierre, tiempo_retiro)
is
    pat = 0;
     $\sigma$  =  $\infty$ ;
    tiempo_abertura = 4;
    tiempo_cierre = 4;
    tiempo_retiro = 2;
end  $S_0$ 

```

```

Y is
  vehiculoHaIngresado_0 :  $\mathbb{N}$ ;
  vehiculoRechazadoSeHaRetirado_1 :  $\mathbb{N}$ ;
end Y

 $\delta_{int}$ (pat,  $\sigma$  , rechazado, tiempo_abertura, tiempo_cierre, tiempo_retiro)
is
  rechazado = false;
  pat = 0;
   $\sigma = \infty$ ;
end  $\delta_{int}$ 

 $\delta_{ext}$ (pat,  $\sigma$  , rechazado, tiempo_abertura, tiempo_cierre, tiempo_retiro,
e, x, port) is
  if port == 0  $\rightarrow$ 
     $\sigma$  = tiempo_abertura + tiempo_cierre + Uniforme(1, 3);
    rechazado = false;
    pat = x;
  if port == 1  $\rightarrow$ 
     $\sigma$  = tiempo_retiro;
    rechazado = true;
    pat = x;
  end  $\delta_{ext}$ 

 $\lambda$ (pat,  $\sigma$  , rechazado, tiempo_abertura, tiempo_cierre, tiempo_retiro)
is
  if not rechazado  $\rightarrow$ 
    vehiculoHaIngresado_0 = pat;
  if rechazado  $\rightarrow$ 
    vehiculoRechazadoSeHaRetirado_1 = pat;
  end  $\lambda$ 

ta(pat,  $\sigma$  , rechazado, tiempo_abertura, tiempo_cierre, tiempo_retiro)
is
   $\sigma$ 
end ta

end atomic

```


3.2.5. BarreraSalida

```
atomic barrera_salida <X, S, S0, Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

  X is
    permitirSalida_0 :  $\mathbb{N}$ ;
  end X

  S is
     $\sigma$  :  $\mathbb{R}$ ;
    pat :  $\mathbb{N}$ ;
    tiempo_abertura :  $\mathbb{R}$ ;
    tiempo_cierre :  $\mathbb{R}$ ;
  end S

  S0 (pat,  $\sigma$  , tiempo_abertura, tiempo_cierre) is
    pat = 0;
     $\sigma$  =  $\infty$ ;
    tiempo_abertura = 4;
    tiempo_cierre = 4;
  end S0

  Y is
    vehiculoHaSalido_0 :  $\mathbb{N}$ ;
  end Y

   $\delta_{int}$ (pat,  $\sigma$  , tiempo_abertura, tiempo_cierre) is
    pat = 0;
     $\sigma$  =  $\infty$ ;
  end  $\delta_{int}$ 

   $\delta_{ext}$ (pat,  $\sigma$  , tiempo_abertura, tiempo_cierre, e, x, port) is
     $\sigma$  = tiempo_abertura + tiempo_cierre + Uniforme(1, 3);
    pat = x;
  end  $\delta_{ext}$ 

   $\lambda$ (pat,  $\sigma$  , tiempo_abertura, tiempo_cierre) is
    vehiculoHaSalido_0 = pat;
  end  $\lambda$ 

  ta(pat,  $\sigma$  , tiempo_abertura, tiempo_cierre) is
     $\sigma$ 
  end ta

end atomic
```

3.2.6. Controlador

```
atomic controlador <X, S, S0, Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

  X is
    vehiculoHaIngresado_0 :  $\mathbb{N}$ ;
    detectarVehiculo_1 :  $\mathbb{N}$ ;
    solicitarSalida_2 :  $\mathbb{N}$ ;
    vehiculoHaSalido_3 :  $\mathbb{N}$ ;
    vehiculoRechazadoSeHaRetirado_4 :  $\mathbb{N}$ ;
  end X

  S is
     $\sigma$  :  $\mathbb{R}$ ;
    pat :  $\mathbb{N}$ ;
    pat_entrada :  $\mathbb{N}$ ;
    cant_autos :  $\mathbb{N}$ ;
    puerto :  $\mathbb{N}$ ;
    pasar_a_barrera_entrada : Boolean;
    esperando_entrar : Boolean;
    tiempo_max_rta_controlador :  $\mathbb{R}$ ;
  end S

  S0 (pat,  $\sigma$ , pat_entrada, cant_autos, puerto, pasar_a_barrera_entrada,
    esperando_entrar, tiempo_max_rta_controlador) is
    pat = 0;
     $\sigma$  =  $\infty$ ;
    pat_entrada = 0;
    cant_autos = 0;
    puerto = 0;
    pasar_a_barrera_entrada = false;
    esperando_entrar = false;
    tiempo_max_rta_controlador = 3;
  end S0

  Y is
    permitirEntradaDirecto_0 :  $\mathbb{N}$ ;
    permitirEntradaEstacionamientoYVolverASensarEntradaNormal_1 :  $\mathbb{N}$ ;
    denegarEntrada_2 :  $\mathbb{N}$ ;
    volverASensarSalidaNormal_3 :  $\mathbb{N}$ ;
    permitirSalida_4 :  $\mathbb{N}$ ;
    permitirEntradaDemoraYVolverASensarSalidaDemora_5 :  $\mathbb{N}$ ;
    volverASensarEntradaRechazado_6 :  $\mathbb{N}$ ;
  end Y
```

```

 $\delta_{int}$ (pat,  $\sigma$  , pat_entrada, cant_autos, puerto, pasar_a_barrera_entrada,
esperando_entrar, tiempo_max_rta_controlador) is
    pasar_a_barrera_entrada = false;
     $\sigma = \infty$ ;
end  $\delta_{int}$ 

```

```

 $\delta_{ext}$ (pat,  $\sigma$  , pat_entrada, cant_autos, puerto, pasar_a_barrera_entrada,
esperando_entrar, tiempo_max_rta_controlador, e, x, port) is
    puerto = port;
    pat = x;
    if port == 0 →
         $\sigma = 0.0$ ;
    if port == 1 →
        if cant_autos < 30 →
            cant_autos = cant_autos + 1;
             $\sigma = 0.0$ ;
        if cant_autos == 30 →
             $\sigma = \text{tiempo\_max\_rta\_controlador}$ ;
            pat_entrada = x;
            esperando_entrar = true;
    if port == 2 →
         $\sigma = 0.0$ ;
    if port == 3 →
        cant_autos = cant_autos - 1;
        if esperando_entrar →
            pat = pat_entrada;
            cant_autos = cant_autos + 1;
            pasar_a_barrera_entrada = true;
            esperando_entrar = false;
         $\sigma = 0.0$ ;
    if port == 4 →
         $\sigma = 0.0$ ;
        esperando_entrar = false;
end  $\delta_{ext}$ 

```

```

 $\lambda$ (pat,  $\sigma$  , pat_entrada, cant_autos, puerto, pasar_a_barrera_entrada,
esperando_entrar, tiempo_max_rta_controlador) is
    if puerto == 0 →
        permitirEntradaEstacionamientoYVolverASensarEntradaNormal_1 =
pat;
    if puerto == 1  $\wedge$  not esperando_entrar →
        permitirEntradaDirecto_0 = pat;
    if puerto == 1  $\wedge$  esperando_entrar →
        esperando_entrar = false;

```

```

        denegarEntrada_2 = pat_entrada;
    if puerto == 2 →
        permitirSalida_4 = pat;
    if puerto == 3 ∧ not pasar_a_barrera_entrada →
        volverASensarSalidaNormal_3 = pat;
    if puerto == 3 ∧ pasar_a_barrera_entrada →
        esperando_entrar = false;
        permitirEntradaDemoraYVolverASensarSalidaDemora_5 = pat;
    if puerto == 4 →
        volverASensarEntradaRechazado_6 = pat;
    end λ

    ta(pat, σ , pat_entrada, cant_autos, puerto, pasar_a_barrera_entrada,
    esperando_entrar, tiempo_max_rta_controlador) is
        σ
    end ta

end atomic

```

3.2.7. Estacionamiento

```

atomic estacionamiento <X, S, S0, Y, δint, δext, λ, ta>is
    X is
        permitirEntrada_0 : N;
    end X

    S is
        σ : ℝ ;
        cola_prioridad : PriorityQueue<Pair< N , ℝ
>MenorMayorConClaveLaSegundaComponente;
        tmp_clone_cola : PriorityQueue<Pair< N , ℝ
>MenorMayorConClaveLaSegundaComponente;
        tmp_new_pair : Pair< N , ℝ >;
        tmp_patente : N ;
        tmp_tiempo : ℝ ;
    end S

    S0 (σ, cola_prioridad, tmp_clone_cola, tmp_new_pair, tmp_patente,
    tmp_tiempo) is
        σ = ∞;
        cola_prioridad = new PriorityQueue(new Pair<>())
    end S0

    Y is
        vehiculoQuiereSalir_0 : N;
    end Y

```

```

     $\delta_{int}$  ( $\sigma$ , cola_prioridad, tmp_cloneCola, tmp_new_pair, tmp_patente,
tmp_tiempo) is
    if not cola_prioridad.empty()  $\rightarrow$ 
        actualizarTiempos(tsim, cola_prioridad, tmp_cloneCola,
tmp_new_pair, tmp_patente, tmp_tiempo);
         $\sigma$  = cola_prioridad.top().second;
    if cola_prioridad.empty()  $\rightarrow$ 
         $\sigma$  =  $\infty$ ;
    end  $\delta_{int}$ 

     $\delta_{ext}$  ( $\sigma$ , cola_prioridad, tmp_cloneCola, tmp_new_pair, tmp_patente,
tmp_tiempo, e, x, port) is
        actualizarTiempos(e, cola_prioridad, tmp_cloneCola, tmp_new_pair,
tmp_patente, tmp_tiempo);
        tmp_new_pair = new Pair<N, R>(x, Uniforme(120, 300));
        cola_prioridad.push(tmp_new_pair);
         $\sigma$  = cola_prioridad.top().second;
    end  $\delta_{ext}$ 

     $\lambda$  ( $\sigma$ , cola_prioridad, tmp_cloneCola, tmp_new_pair, tmp_patente,
tmp_tiempo) is
        tmp_patente = cola_prioridad.top().first;
        cola_prioridad.pop();
        vehiculoQuiereSalir_0 = tmp_patente;
    end  $\lambda$ 

    ta( $\sigma$ , cola_prioridad) is
         $\sigma$ ;
    end ta

    actualizarTiempos(e, cola_prioridad, tmp_cloneCola, tmp_new_pair,
tmp_patente, tmp_tiempo) is
        foreach par_auto in cola_prioridad do
            tmp_patente = cola_prioridad.top().first;
            tmp_tiempo = cola_prioridad.top().second - e;
            cola_prioridad.pop();
            if (tmp_tiempo < 0.0)  $\rightarrow$  tmp_tiempo = 0.0;
            tmp_new_pair = new Pair<N, R>(tmp_patente, tmp_tiempo);
            tmp_cloneCola.push(tmp_new_pair);
        end foreach
        cola_prioridad = tmp_cloneCola;
    end actualizarTiempos
end atomic

```

3.2.8. Monitor

```
atomic monitor <X, S, S0, Y,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta>is

  X is
    barreraEntradaAdmitidos_0 :  $\mathbb{N}$ ;
    barreraEntradaRechazados_1 :  $\mathbb{N}$ ;
    barreraSalidaEgresoAbsoluto_2 :  $\mathbb{N}$ ;
  end X

  S is
     $\sigma$  :  $\mathbb{R}$ ;
    tiempos_entrada, tiempos_salida : List< $\mathbb{R}$ >;
    autos_entrantes, autos_salientes : List< $\mathbb{N}$ >;
    K_ESTACIONAMIENTO, total_intentos, rechazados, total_admitidos,
total_salidos,
    tiempo_ultimo_evento, ocupacion_actual, suma_tiempos_permanencia,
acumulado_ocupacion,
    promedio_permanencia, tasa_rechazo, ocupacion_promedio :  $\mathbb{R}$ ;
    propiedad_safety, propiedad_liveness_1 : Boolean;
    ultimo_puerto :  $\mathbb{N}$ ;
  end S

  S0 (tiempos_entrada, tiempos_salida, autos_entrantes, autos_salientes,
K_ESTACIONAMIENTO, total_intentos, rechazados, total_admitidos,
total_salidos, tiempo_ultimo_evento, ocupacion_actual,
suma_tiempos_permanencia, acumulado_ocupacion, promedio_permanencia,
tasa_rechazo, ocupacion_promedio, propiedad_safety, propiedad_liveness_1,
ultimo_puerto) is

     $\sigma$  =  $\infty$ ;
    tiempos_entrada = [];
    tiempos_salida = [];
    autos_entrantes = [];
    autos_salientes = [];
    K_ESTACIONAMIENTO = 30;
    total_intentos = 0;
    rechazados = 0;
    total_admitidos = 0;
    total_salidos = 0;
    tiempo_ultimo_evento = 0;
    ocupacion_actual = 0;
    suma_tiempos_permanencia = 0;
    acumulado_ocupacion = 0;
    promedio_permanencia = 0;
    tasa_rechazo = 0;
```

```

    ocupacion_promedio = 0;
    propiedad_safety = true;
    propiedad_liveness_1 = true;
    ultimo_puerto = 0;
end S0

Y is
    tiempoPromedioPermanencia_0 : ℝ;
    tasaRechazo_1 : ℝ;
    ocupacionPromedio_2 : ℝ;
end Y

 $\delta_{int}$ (tiempos_entrada, tiempos_salida, autos_entrantes, autos_salientes,
K_ESTACIONAMIENTO, total_intentos, rechazados, total_admitidos,
total_salidos, tiempo_ultimo_evento, ocupacion_actual,
suma_tiempos_permanencia, acumulado_ocupacion, promedio_permanencia,
tasa_rechazo, ocupacion_promedio, propiedad_safety, propiedad_liveness_1,
ultimo_puerto) is
     $\sigma = \infty$ ;
end  $\delta_{int}$ 

 $\delta_{ext}$ (tiempos_entrada, tiempos_salida, autos_entrantes, autos_salientes,
K_ESTACIONAMIENTO, total_intentos, rechazados, total_admitidos,
total_salidos, tiempo_ultimo_evento, ocupacion_actual,
suma_tiempos_permanencia, acumulado_ocupacion, promedio_permanencia,
tasa_rechazo, ocupacion_promedio, propiedad_safety, propiedad_liveness_1,
ultimo_puerto, e, x, port) is
    ultimo_puerto = port;
    acumulado_ocupacion += ocupacion_actual * (e -
tiempo_ultimo_evento);
    tiempo_ultimo_evento = e;
    if port == 0 →
        total_intentos += 1;
        total_admitidos += 1;
        ocupacion_actual += 1;
        (resize y asignación de tiempos y autos entrantes);
        propiedad_safety = propiedad_safety  $\wedge$  (ocupacion_actual  $\leq$ 
K_ESTACIONAMIENTO);
         $\sigma = 0.0$ ;
    if port == 1 →
        total_intentos += 1;
        rechazados += 1;
        tasa_rechazo = rechazados / total_intentos;
        propiedad_safety = propiedad_safety  $\wedge$  (ocupacion_actual  $\leq$ 
K_ESTACIONAMIENTO);

```

```

         $\sigma = 0.0$ ;
    if port == 2  $\rightarrow$ 
        (resize y asignación de tiempos y autos salientes);
        suma_tiempos_permanencia += tiempos_salida[patente] -
tiempos_entrada[patente];
        total_salidos += 1;
        promedio_permanencia = suma_tiempos_permanencia /
total_salidos;
        ocupacion_actual -= 1;
        if e > 0  $\rightarrow$  ocupacion_promedio = acumulado_ocupacion / e;
        propiedad_safety = propiedad_safety  $\wedge$  (ocupacion_actual  $\leq$ 
K_ESTACIONAMIENTO);
        actualizarPropiedadLiveness1(e, x, ...);
         $\sigma = 0.0$ ;
    end  $\delta_{ext}$ 

 $\lambda(\sigma)$  is
    if ultimo_puerto == 0  $\rightarrow$  tiempoPromedioPermanencia_0 =
promedio_permanencia;
    if ultimo_puerto == 1  $\rightarrow$  tasaRechazo_1 = tasa_rechazo;
    if ultimo_puerto == 2  $\rightarrow$  ocupacionPromedio_2 = ocupacion_promedio;
    end  $\lambda$ 

ta( $\sigma$ ) is
     $\sigma$ 
end ta

actualizarPropiedadLiveness1(e, patente, autos_entrantes,
autos_salientes, propiedad_liveness_1) is
    bool entro = false;
    for i in 0 .. autos_entrantes.size()  $\rightarrow$ 
        if patente == autos_entrantes[i]  $\rightarrow$  entro = true; break;
    bool salio = false;
    for i in 0 .. autos_salientes.size()  $\rightarrow$ 
        if patente == autos_salientes[i]  $\rightarrow$  salio = true; break;
    propiedad_liveness_1 = propiedad_liveness_1  $\wedge$  (entro  $\wedge$  salio);
    end actualizarPropiedadLiveness1

end atomic

```


4. Implementación en PowerDEVS

En la herramienta PowerDEVS, el tiempo de simulación que se maneja internamente no tiene una unidad de tiempo especificada, así que nosotros decidimos que serán segundos.

Así entonces, toda relación con el tiempo en el código del sistema de simulación se verá reflejado en segundos.

4.1. Componentes parametrizables

Los componentes parametrizables al implementar el sistema en PowerDEVS son:

- **GeneradorArribos**
 - mean: La media deseada para el generador aleatorio
 - seed: La semilla deseada para el generador aleatorio
- **SensorEntrada**
 - valor: Es un valor numérico que si es distinto de cero entonces el sensor inicia bloqueado
 - latencia: Lo que demora el sensor en detectar un auto
 - TIEMPO_MAX_ENTRADA_VEHICULOS: Tiempo máximo para que entren autos al estacionamiento
- **SensorSalida**
 - valor: Un valor numérico que si es distinto de cero, el sensor inicia bloqueado
 - latencia: Lo que demora el sensor en detectar un auto
- **BarreraEntrada**
 - seed: La semilla deseada para la distribución uniforme
 - a: El valor mínimo de la distribución uniforme
 - b: El valor máximo de la distribución uniforme
 - tiempo_apertura: Lo que demora la barrera en abrirse
 - tiempo_cierre: Lo que demora la barrera en cerrarse
 - tiempo_retiro: Lo que demora un auto rechazado en retirarse
- **BarreraSalida**
 - seed: La semilla deseada para la distribución uniforme
 - a: El valor mínimo de la distribución uniforme
 - b: El valor máximo de la distribución uniforme
 - tiempo_apertura: Lo que demora la barrera en abrirse

- tiempo_cierre: Lo que demora la barrera en cerrarse
- Controlador
 - capacidad_max_estacionamiento: Un valor numérico referido a la capacidad máxima del estacionamiento
 - tiempo_max_rta_controlador: Tiempo de decisión máximo del controlador
- Monitor
 - K_ESTACIONAMIENTO: Un valor numérico referido a la capacidad máxima del estacionamiento

4.2. Conexión de modelos

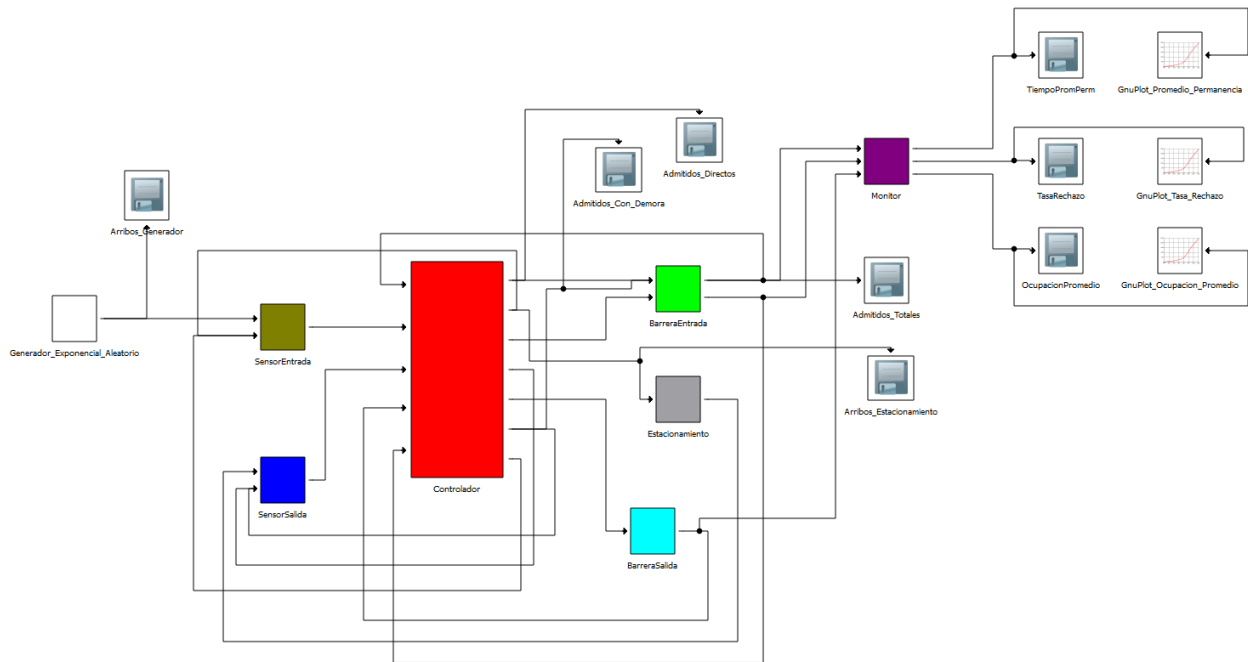


Figura 9: Modelo de simulación implementado en PowerDevs

4.3. Configuración de la simulación

La implementación tiene la configuración referida al Cuadro 2

- Duración:
 - Tiempo máximo de simulación:
Se configura al ejecutar la simulación, en el campo **Final Time**
 - Tiempo máximo del estacionamiento:
Se configura en los parámetros del sensor de entrada
- Parámetros de los distintos atomicos:
Para saber cómo ajustar los parámetros de los atomicos, leer la subsección **Componentes parametrizables**

4.4. Tabla de parámetros principales

Atomic	Parámetro	Valor	Descripción
GeneradorArribos	mean	10	Media para los interarribos
GeneradorArribos	seed	3	Semilla para el generador exponencial
SensorEntrada	valor	0	Boolean de sensor bloqueado
SensorEntrada	latencia	1	Latencia en sensar un auto
SensorEntrada	TIEMPO_MAX_ENTRADA_VEHICULOS	43200	Tiempo máximo que está abierto el estacionamiento
SensorSalida	valor	0	Boolean de sensor bloqueado
SensorSalida	latencia	1	Latencia en sensar un auto
BarreraEntrada	seed	29	Semilla para la distribución uniforme
BarreraEntrada	a	1	Mínimo valor de la distribución uniforme
BarreraEntrada	b	3	Máximo valor de la distribución uniforme
BarreraEntrada	tiempo_apertura	4	Tiempo que tarda la barrera en subir
BarreraEntrada	tiempo_cierre	4	Tiempo que tarda la barrera en bajar
BarreraEntrada	tiempo_retiro	2	Tiempo que tarda un auto rechazado en retirarse de la barrera
BarreraSalida	seed	29	Semilla para la distribución uniforme
BarreraSalida	a	1	Mínimo valor de la distribución uniforme
BarreraSalida	b	3	Máximo valor de la distribución uniforme
BarreraSalida	tiempo_apertura	4	Tiempo que tarda la barrera en subir
BarreraSalida	tiempo_cierre	4	Tiempo que tarda la barrera en bajar
Controlador	capacidad_max_estacionamiento	30	Capacidad máxima del estacionamiento
Controlador	tiempo_max_rta_controlador	3	Tolerancia sobre autos con posibilidad de rechazo
Monitor	K_ESTACIONAMIENTO	30	Capacidad máxima del estacionamiento

Cuadro 3: Tabla con los valores de los parámetros usados en los distintos atomics

5. Diseño Experimental y Métricas

5.1. Métricas a capturar

- Tiempo medio de permanencia (desde llegada hasta egreso).
- Tasa de rechazo: vehículos denegados / llegadas totales.
- Ocupación promedio: integral de ocupación / tiempo total.

6. Casos propuestos para las corridas:

N.º	Nombre	Parámetros modificados	Objetivo del experimento
1	Caso Sugerido (dado por el informe)	Capacidad = 30 Arribos $\sim \text{Exp}(\lambda = 1/10 \text{ s})$ Barreras: abrir 4 s, cerrar 4 s, pasar_barrera $\sim \text{U}(1, 3)$ Permanencia $\sim \text{U}(120, 300 \text{ s})$	Establecer línea base para comparación
2	Alta demanda	Arribos $\sim \text{Exp}(\lambda = 1/5 \text{ s})$	Evaluar saturación del sistema y tasa de rechazo
3	Menor capacidad	Capacidad = 10	Estudiar impacto en liveness y rechazos
4	Mayor capacidad	Capacidad = 50 Arribos $\sim \text{Exp}(\lambda = 1/3 \text{ s})$ Barrera: abrir/cerrar = 2 s	Evaluar estabilidad y ocupación en condiciones relajadas
5	Demora en barreras	Barrera: abrir/cerrar = 8 s	Observar demoras acumuladas y posibles colas
6	Barrera rápida	Barrera: abrir/cerrar = 2 s	Medir mejora en rendimiento
7	Permanencia alta	Permanencia $\sim \text{U}(300, 600 \text{ s})$	Simular vehículos que permanecen más tiempo estacionados
8	Permanencia baja	Permanencia $\sim \text{U}(60, 120 \text{ s})$	Simular alta rotación de vehículos
9	Sensores más lentos	latencia = 3	Analizar el comportamiento de las colas
10	Congestión total	Capacidad = 10 Arribos $\sim \text{Exp}(\lambda = 1/4 \text{ s})$ Permanencia $\sim \text{U}(300, 600 \text{ s})$	Evaluar comportamiento extremo ante bloqueos

Cuadro 4: Configuraciones experimentales para la simulación del estacionamiento

7. Experimentos, Resultados y Gráficos

Cada gráfico representa:

- **Ocupación promedio:** cantidad acumulada de vehículos dentro del estacionamiento a lo largo del tiempo.
- **Permanencia promedio:** histograma del tiempo que cada vehículo permaneció dentro del estacionamiento.
- **Tasa de rechazo:** cantidad de autos rechazados a lo largo de la simulación sobre la cantidad de autos que intentaron entrar (admitidos+rechazados).

El eje coordenado **x** denota el Tiempo de la simulación, expresado en segundos.

El eje coordenado **y** denota el valor del estadístico que corresponda, en un tiempo determinado.

7.1. Experimento 1

Véase configuración en la Tabla 6

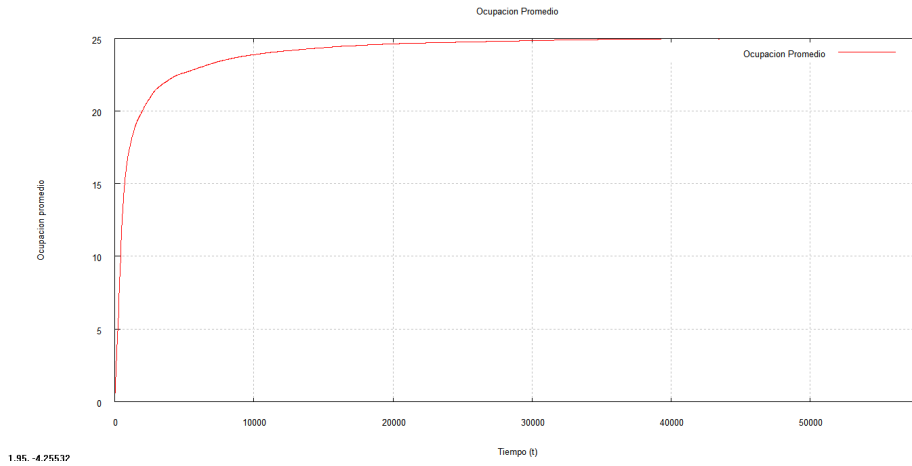


Figura 10: Ocupación promedio del Experimento 1

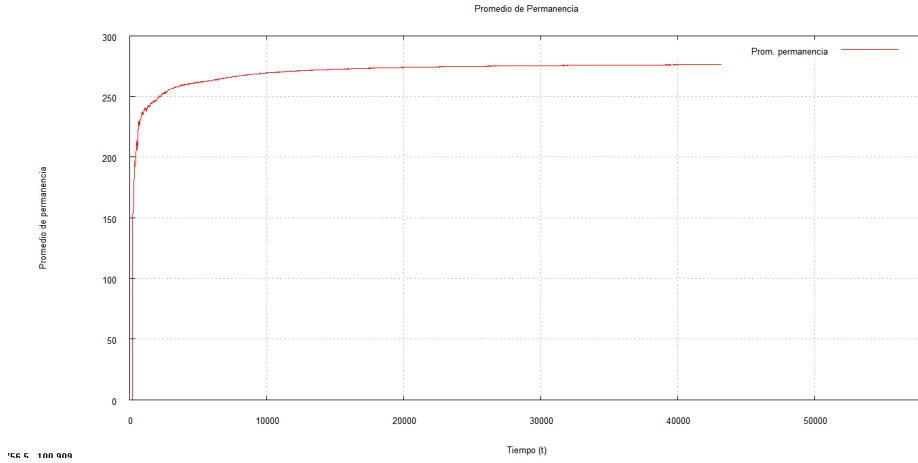


Figura 11: Tiempos de permanencia del Experimento 1

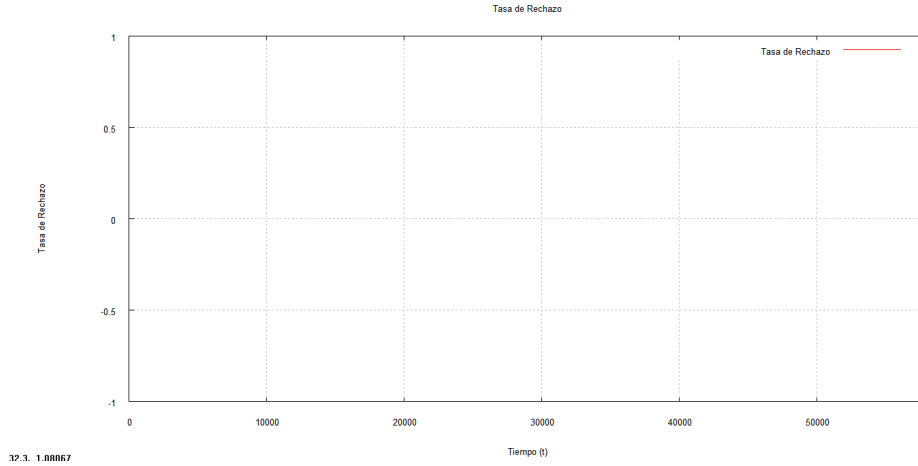


Figura 12: Rechazos acumulados del Experimento 1

Conclusión del Experimento 1: El controlador no rechazó ningún vehículo; la ocupación se mantuvo en torno a 25 autos de una capacidad máxima de 30, y los tiempos de permanencia oscilaron entre 250 y 300 unidades de tiempo.

7.2. Experimento 2

Véase configuración en la Tabla 6

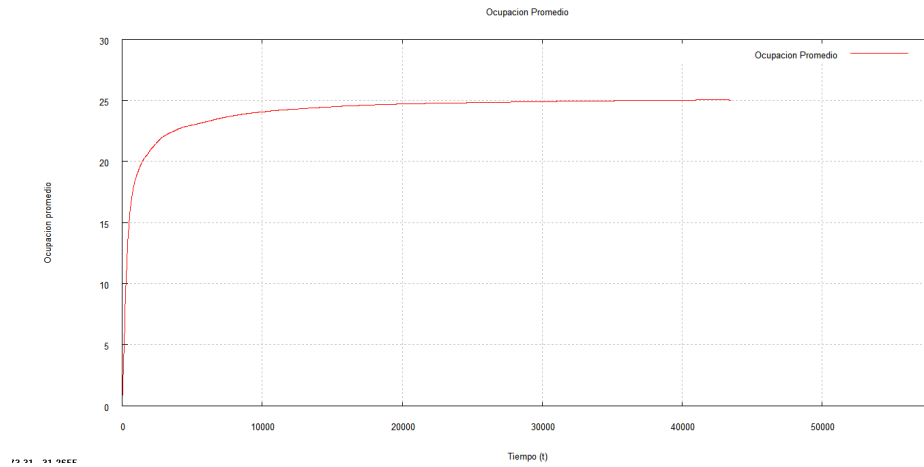


Figura 13: Ocupación promedio del Experimento 2

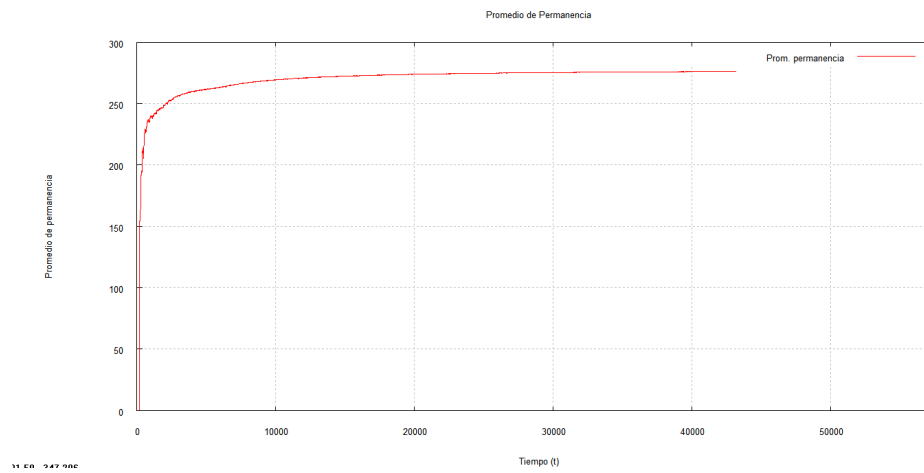


Figura 14: Tiempos de permanencia del Experimento 2

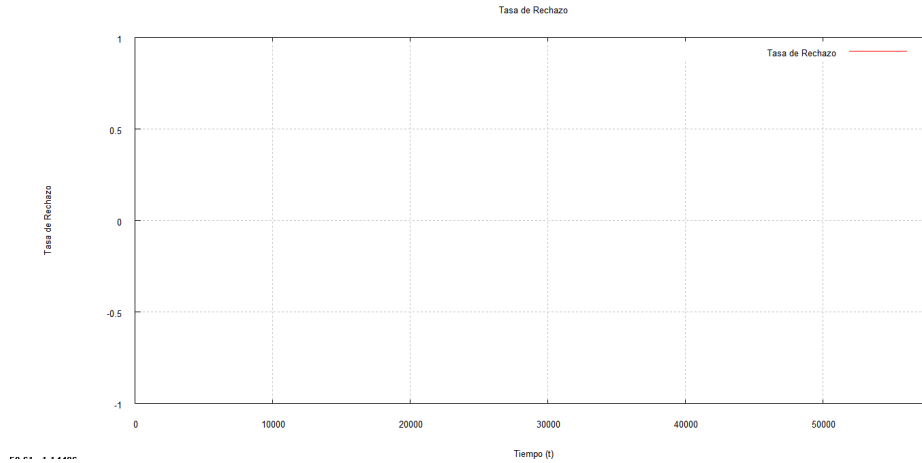


Figura 15: Rechazos acumulados del Experimento 2

Conclusión del Experimento 2: Al igual que en el Experimento 1, no hubo rechazos y la ocupación promedio alcanzó aproximadamente 25 vehículos, con tiempos de permanencia entre 250 y 300. La similitud de resultados evidencia que la barrera de entrada actúa como cuello de botella, permitiendo la salida de vehículos antes de que lleguen nuevos, lo que explica la ausencia de rechazos.

7.3. Experimento 3

Véase configuración en la Tabla 6

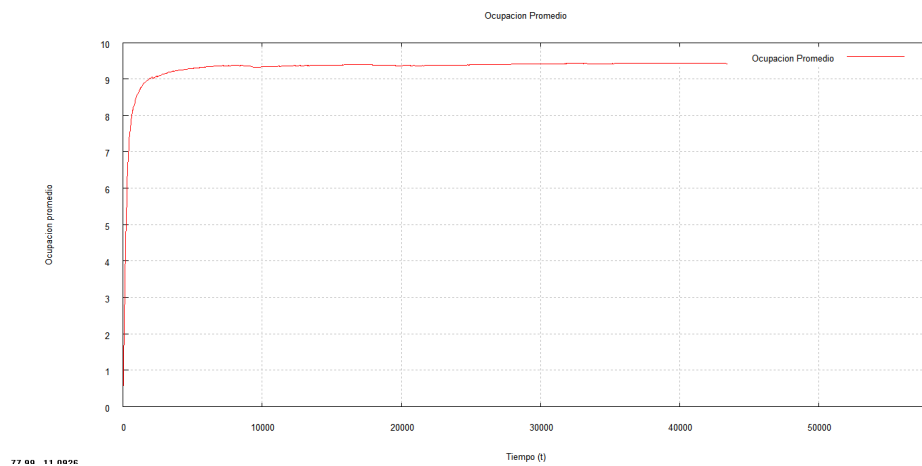


Figura 16: Ocupación promedio del Experimento 3

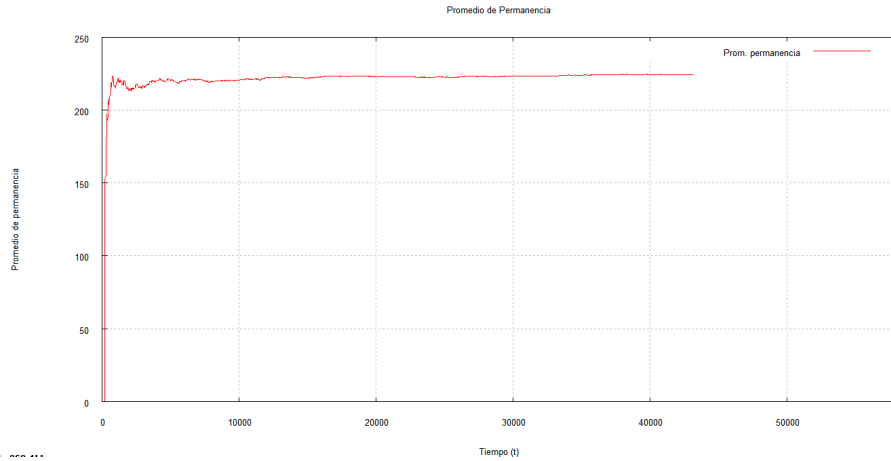


Figura 17: Tiempos de permanencia del Experimento 3

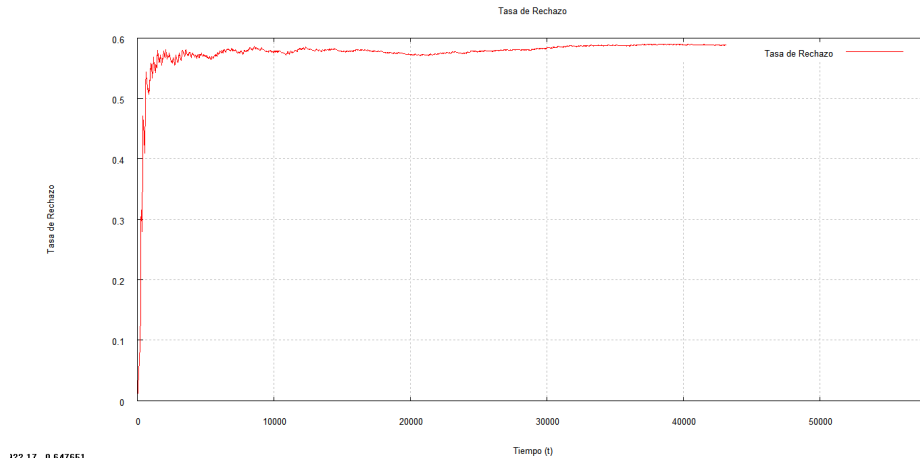


Figura 18: Rechazos acumulados del Experimento 3

Conclusión del Experimento 3: A diferencia de los experimentos previos, en este caso sí se registraron rechazos. Al reducir la capacidad del estacionamiento, la tasa de rechazo aumentó y la ocupación promedio alcanzó el límite máximo; los tiempos de permanencia se mantuvieron entre 250 y 300.

7.4. Experimento 4

Véase configuración en la Tabla 6

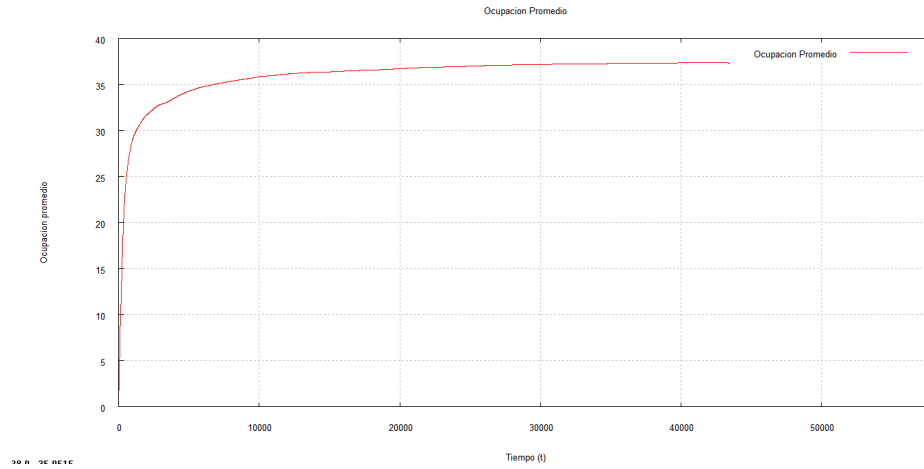


Figura 19: Ocupación promedio del Experimento 4

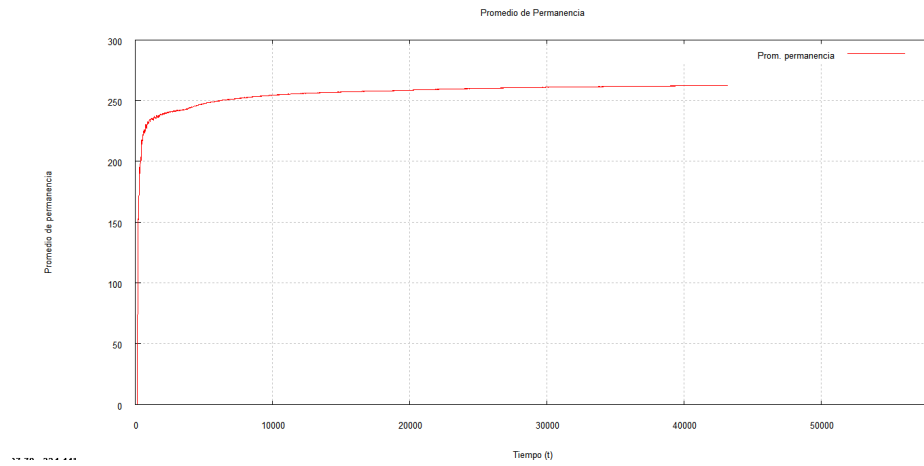


Figura 20: Tiempos de permanencia del Experimento 4

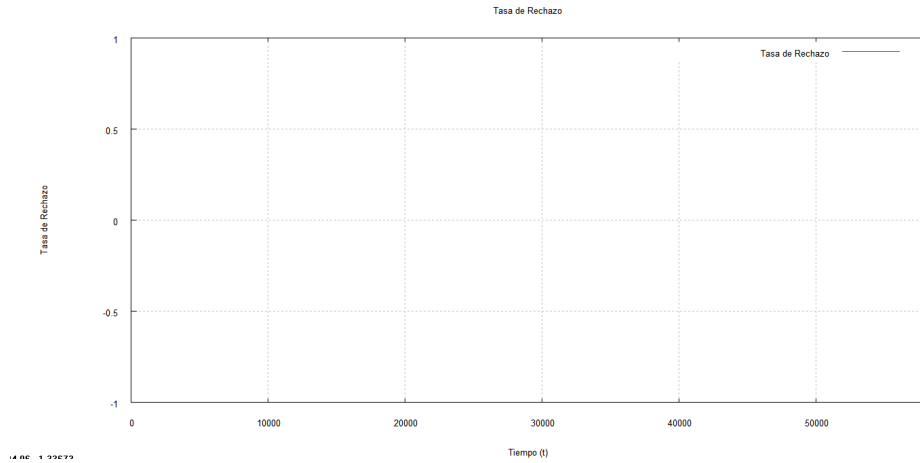


Figura 21: Rechazos acumulados del Experimento 4

Conclusión del Experimento 4: A pesar de un mayor ritmo de arribos, el incremento de la capacidad y la duplicación de la velocidad de las barreras evitaron rechazos; la ocupación promedio se situó entre 35 y 40 vehículos, con tiempos de permanencia entre 250 y 300.

7.5. Experimento 5

Véase configuración en la Tabla 6

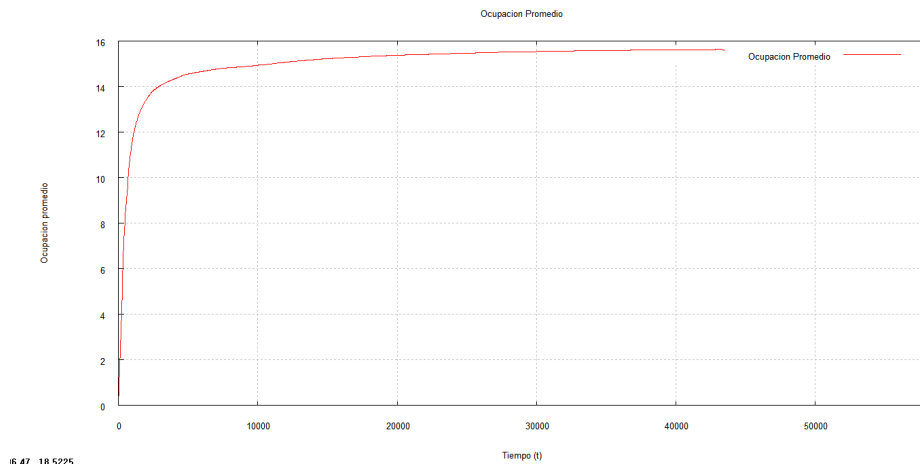


Figura 22: Ocupación promedio del Experimento 5

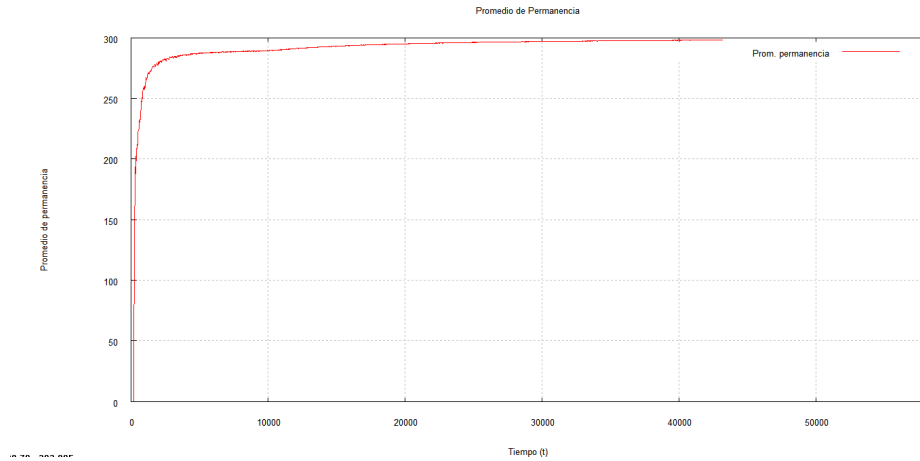


Figura 23: Tiempos de permanencia del Experimento 5

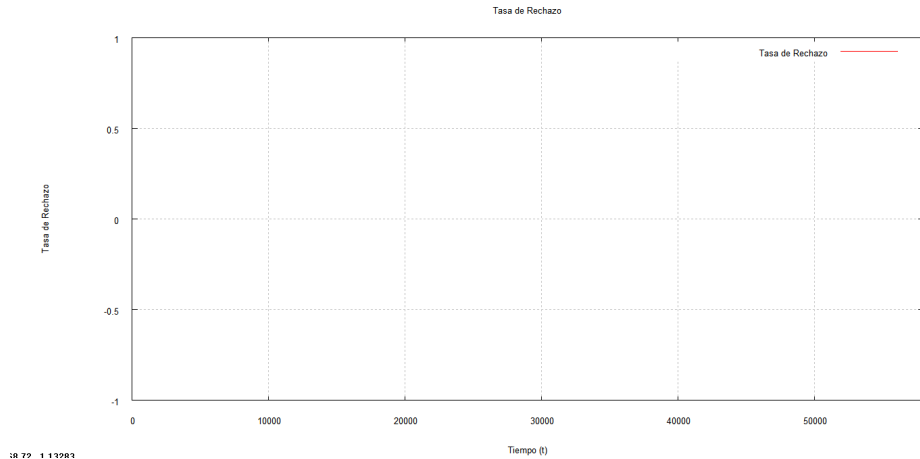


Figura 24: Rechazos acumulados del Experimento 5

Conclusión del Experimento 5: Al aumentar el doble el tiempo de rendimiento de las barreras(más lentas), la ocupación promedio descendió a 14–16 vehículos y los tiempos de permanencia se mantuvieron entre 250 y 300. No se produjeron rechazos, ya que la lentitud de las barreras limitó el número de ingresos.

7.6. Experimento 6

Véase configuración en la Tabla 6

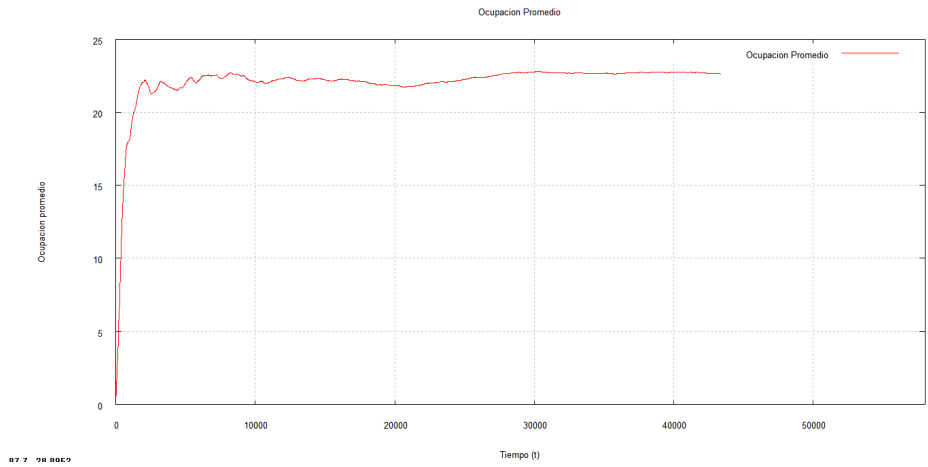


Figura 25: Ocupación promedio del Experimento 6

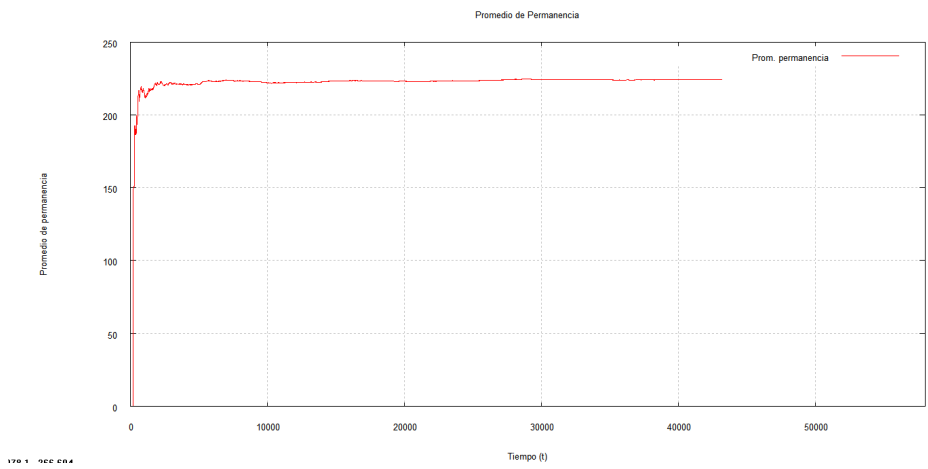


Figura 26: Tiempos de permanencia del Experimento 6

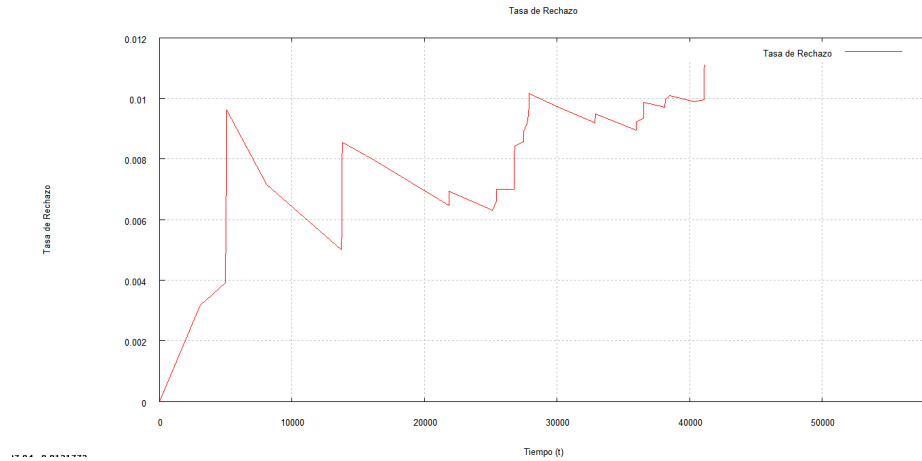


Figura 27: Rechazos acumulados del Experimento 6

Conclusión del Experimento 6: Con las barreras al doble de velocidad, es decir, mejor rendimiento, menor tiempo de demora y las demás condiciones iguales, la ocupación promedio osciló entre 20 y 25 vehículos y los tiempos de permanencia entre 200 y 250; sin embargo, se observaron picos ascendentes en la tasa de rechazo, que alcanzó valores próximos a 0,012, debido a la saturación cuando un vehículo llegaba con el estacionamiento lleno.

7.7. Experimento 7

Véase configuración en la Tabla 6

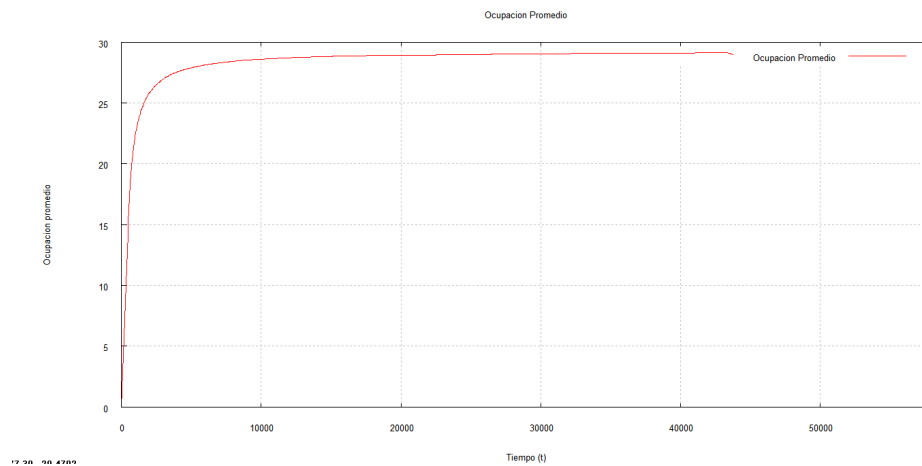


Figura 28: Ocupación promedio del Experimento 7

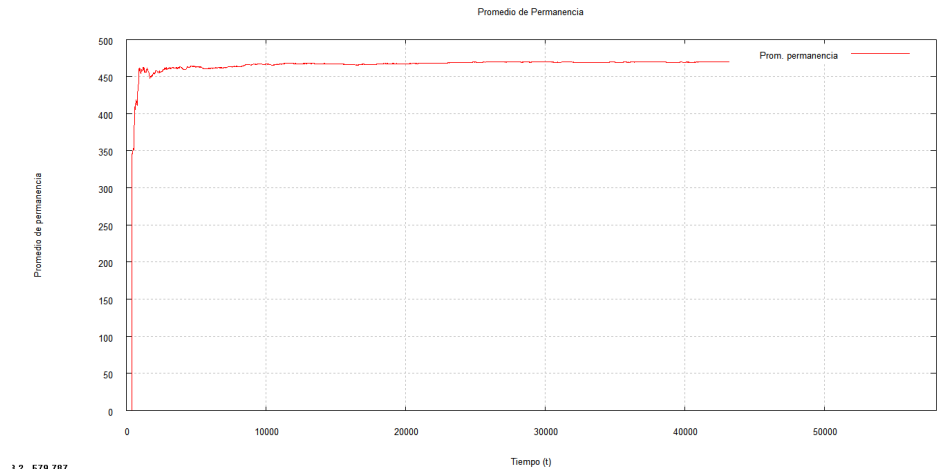


Figura 29: Tiempos de permanencia del Experimento 7

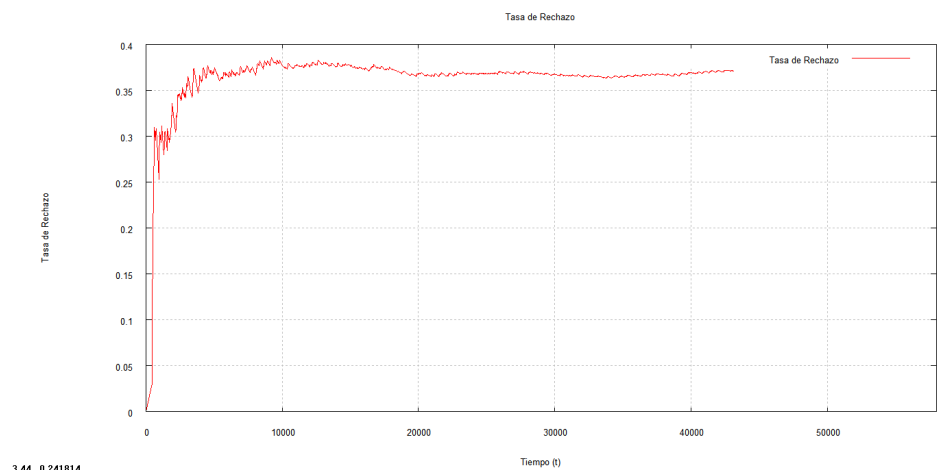


Figura 30: Rechazos acumulados del Experimento 7

Conclusión del Experimento 7: La ocupación promedio se mantuvo al límite de la capacidad (30 vehículos), los tiempos de permanencia subieron a 450–500 y la tasa de rechazo escaló entre 0,35 y 0,4. En escenarios de estancias prolongadas es natural alcanzar la saturación y que aumenten los rechazos, lo que sugiere la necesidad de aumentar recursos, por ejemplo, ampliando la capacidad máxima.

7.8. Experimento 8

Véase configuración en la Tabla 6

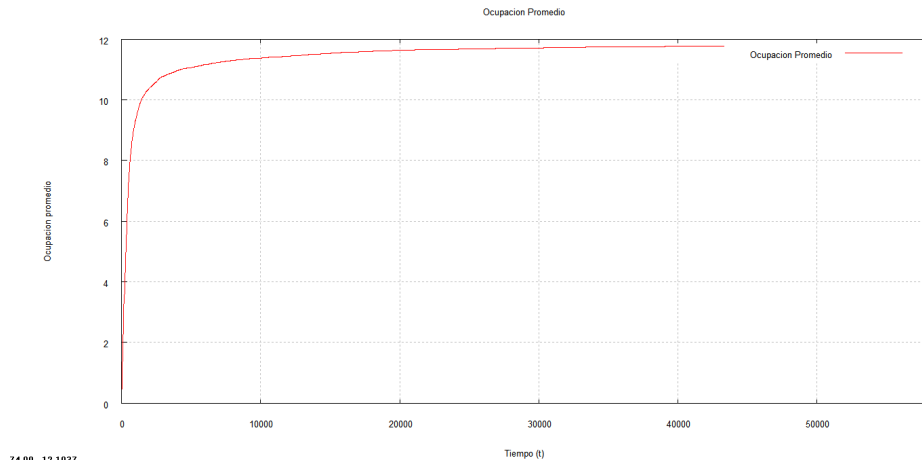


Figura 31: Ocupación promedio del Experimento 8

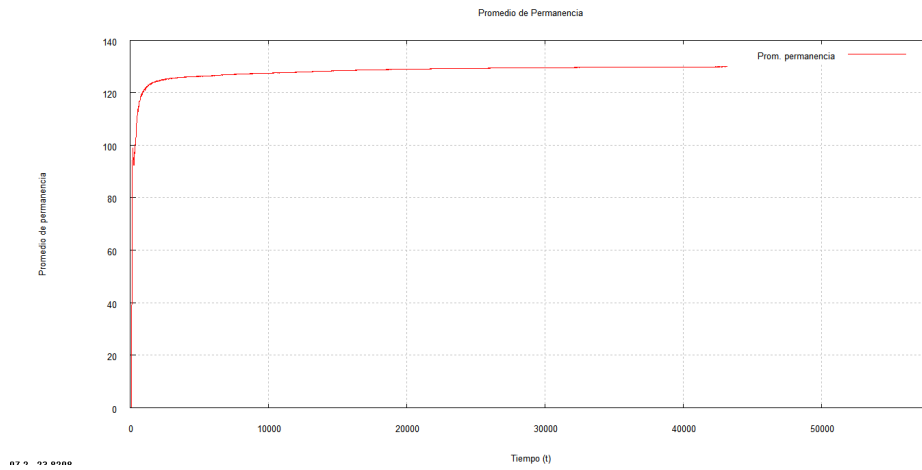


Figura 32: Tiempos de permanencia del Experimento 8

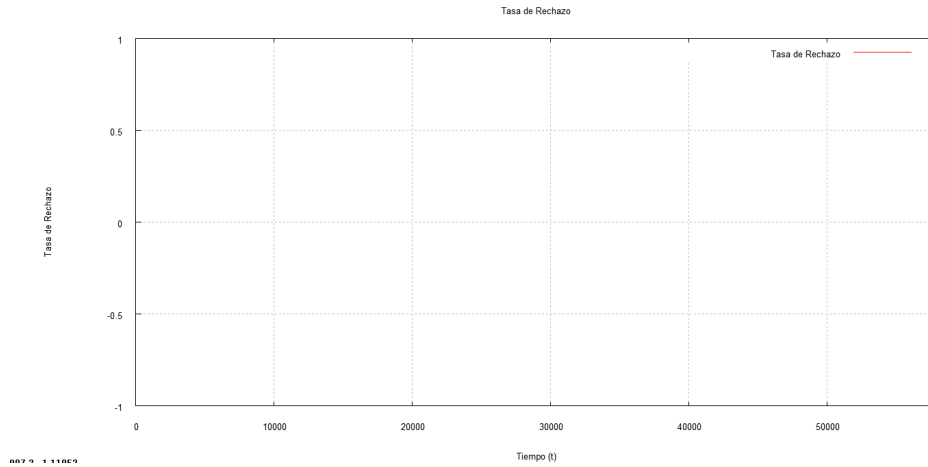


Figura 33: Rechazos acumulados del Experimento 8

Conclusión del Experimento 8: Con tiempos de permanencia reducidos a 120–140, la ocupación promedio descendió a 10–12 vehículos y no se registraron rechazos, confirmando que estadías más breves alivian la carga sin afectar la tasa de rechazo respecto a escenarios de baja demanda.

7.9. Experimento 9

Véase configuración en la Tabla 6

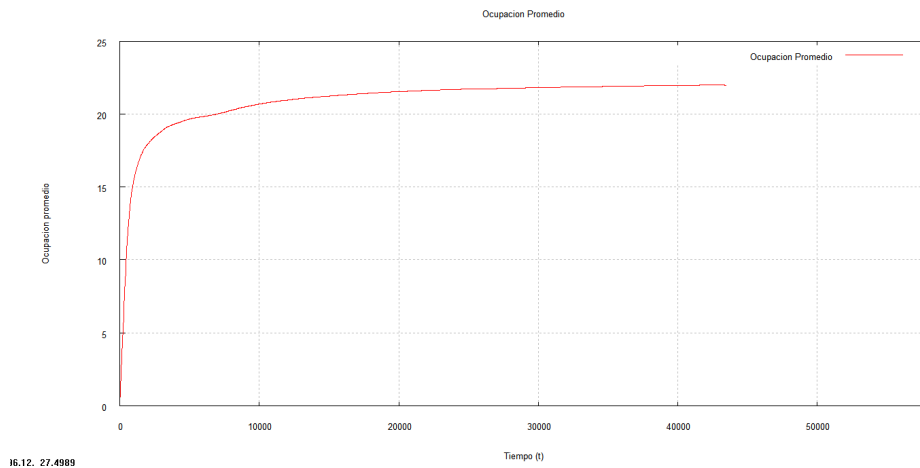


Figura 34: Ocupación promedio del Experimento 9

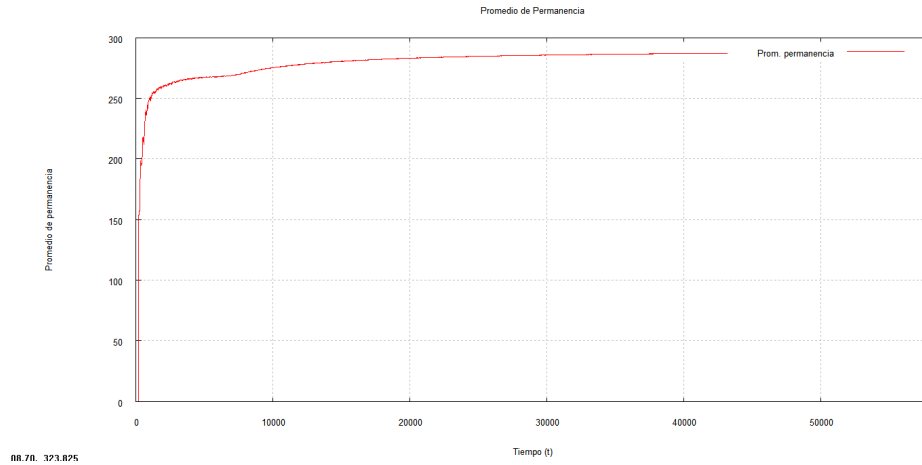


Figura 35: Tiempos de permanencia del Experimento 9

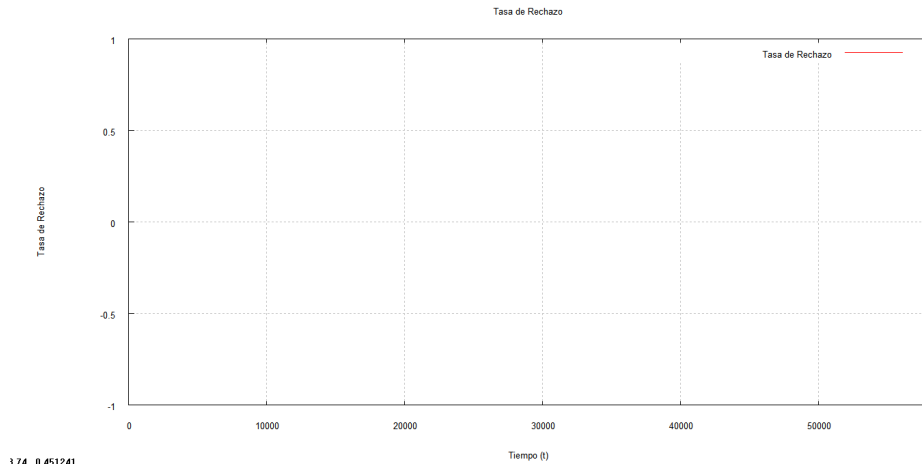


Figura 36: Rechazos acumulados del Experimento 9

Conclusión del Experimento 9: Al ralentizar los sensores, la ocupación promedio se redujo a 20–25 vehículos, con tiempos de permanencia entre 250 y 300 y sin rechazos, lo que demuestra que una detección más lenta limita los ingresos sin provocar rechazos.

7.10. Experimento 10

Véase configuración en la Tabla 6

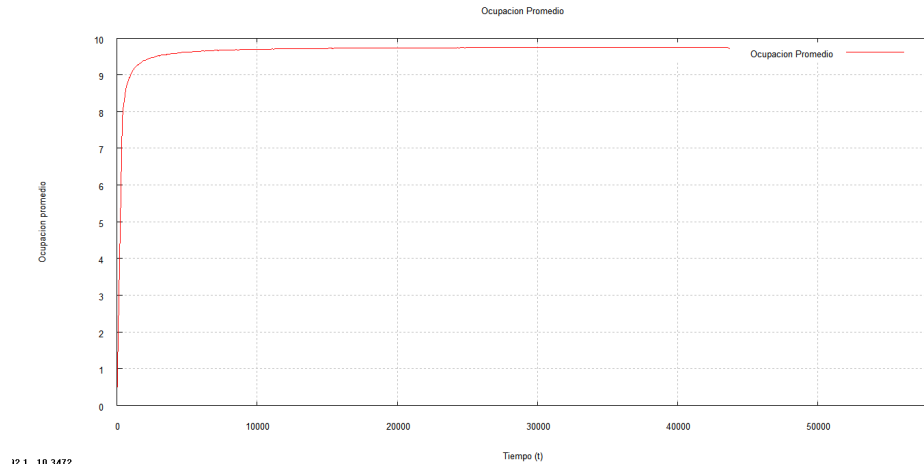


Figura 37: Ocupación promedio del Experimento 10

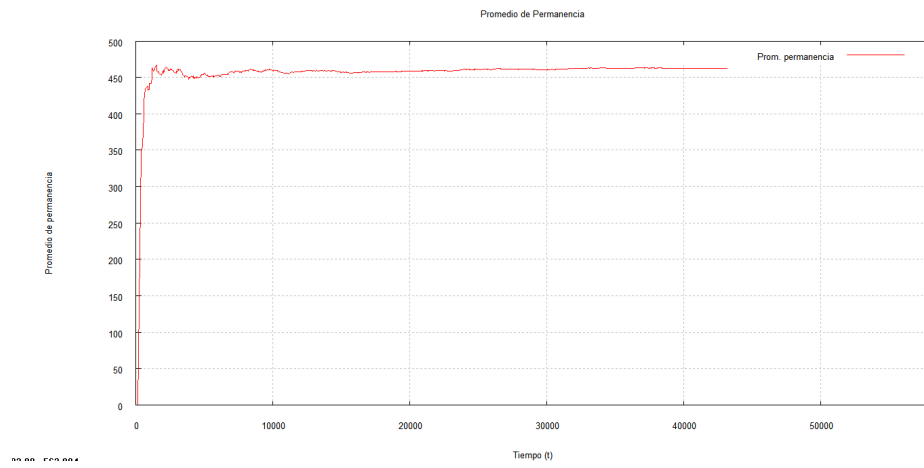


Figura 38: Tiempos de permanencia del Experimento 10

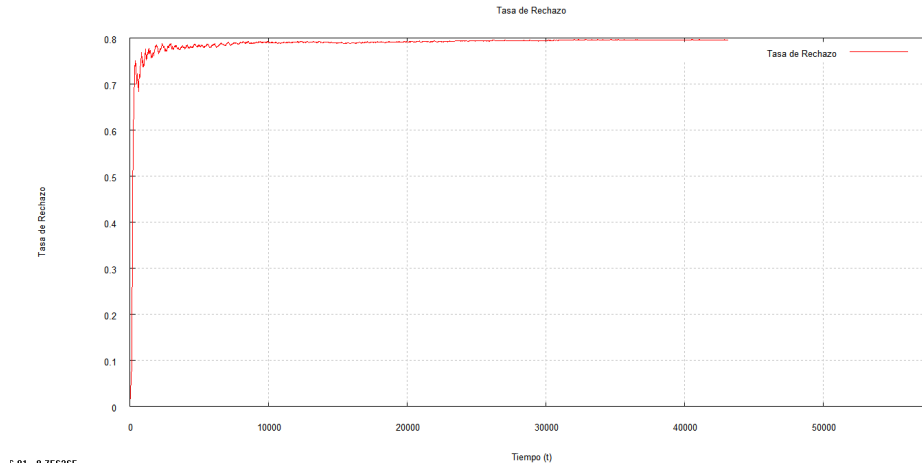


Figura 39: Rechazos acumulados del Experimento 10

Conclusión del Experimento 10: En un escenario muy adverso —capacidad mínima, arribos rápidos y permanencias largas— la ocupación se mantuvo al límite (9–10 vehículos), los tiempos de permanencia ascendieron a 450–500 y la tasa de rechazo aumentó drásticamente entre 0,7 y 0,8, rechazándose casi todos los vehículos al saturarse el sistema desde el inicio.

Conclusión General:

Los experimentos realizados muestran una fuerte dependencia del desempeño del sistema respecto a la configuración de sus parámetros, como la capacidad del estacionamiento, la tasa de arribo de vehículos, y el tiempo de permanencia. En escenarios con baja demanda, el sistema opera eficientemente sin rechazos y con tiempos razonables. En cambio, al aumentar la demanda sin modificar la capacidad, la tasa de rechazo crece rápidamente, y los tiempos de permanencia se vuelven muy elevados, lo cual puede deteriorar la experiencia del usuario y la eficiencia operativa. Estos resultados permiten identificar configuraciones que garantizan seguridad, eficiencia y satisfacción del usuario.

8. Intervalo de confianza

Métrica	Media	Desvío muestral
Tasa de Rechazo	0.1811	0.3095
Tiempo de Permanencia (s)	290.85	103.76
Promedio de Ocupación (vehículos)	20.69	9.05

Cuadro 5: Resumen estadístico de los 10 experimentos

$$IC = \bar{X} \pm t_{1-\alpha/2, n-1} \cdot \frac{S}{\sqrt{n}}$$

donde:

- $\bar{X}$ es la media muestral.
- S es el desvío estándar muestral.
- n es el tamaño de la muestra.

Para $\alpha = 0.05$ y $n = 10$:

$$t_{1-0.025, 9} = t_{0.975, 9} \approx 2.2622$$

8.1. Intervalo de confianza del 95 % para la Tasa de Rechazo

$$\bar{X} = 0,1811, \quad S = 0,3095$$

$$\begin{aligned} IC_{95\%} &= 0.1811 \pm 2.2622 \cdot \frac{0.3095}{\sqrt{10}} \approx 0.1811 \pm 0.2214 \\ &\Rightarrow [-0.0403, 0.4025] \end{aligned}$$

8.2. Intervalo de confianza del 95 % para el Tiempo de Permanencia

$$\bar{X} = 290,85, \quad S = 103,76$$

$$\begin{aligned} IC_{95\%} &= 290.85 \pm 2.2622 \cdot \frac{103.76}{\sqrt{10}} \approx 290.85 \pm 74.20 \\ &\Rightarrow [216.6231, 365.0768] \end{aligned}$$

8.3. Intervalo de confianza del 95 % para la Ocupación Promedio

$$\bar{X} = 20.69, \quad S = 9.05$$

$$IC_{95\%} = 20.69 \pm 2.2622 \cdot \frac{9.05}{\sqrt{10}} \approx 20.69 \pm 6.47$$

$$\Rightarrow [14.2159, 27.1641]$$

8.4. Visualización de los intervalos de confianza

A continuación se presenta un gráfico de barras de error (*error bar plot*) que muestra los intervalos de confianza del 95 % para las tres métricas principales evaluadas en los experimentos:

- Tasa de Rechazo
- Tiempo Promedio de Permanencia
- Ocupación Promedio

Cada punto central representa la media observada de la métrica, mientras que las barras verticales indican el intervalo de confianza asociado. Estas barras permiten visualizar gráficamente la incertidumbre en la estimación de la media, basada en la variabilidad de los datos obtenidos en las distintas simulaciones.

Este tipo de representación resulta útil para:

- Comparar la precisión relativa de las estimaciones.
- Evaluar si las diferencias entre métricas podrían ser estadísticamente significativas.
- Detectar posibles solapamientos entre intervalos que sugieran comportamientos similares.

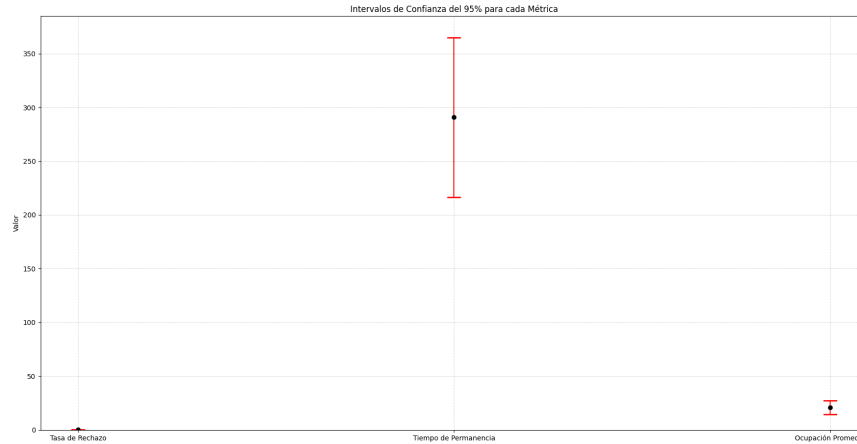


Figura 40: Intervalos de confianza del 95 % para las métricas clave del sistema de estacionamiento: tasa de rechazo, tiempo promedio de permanencia y ocupación promedio. Cada barra representa el intervalo calculado. *Nota: el intervalo de confianza para la tasa de rechazo incluye valores negativos debido a la dispersión y baja media. Este resultado carece de sentido práctico (una tasa negativa no es posible), pero se conserva para reflejar la variabilidad estadística de la muestra.*

Para más detalles sobre los valores utilizados, véase las secciones de los intervalos:

- Tasa de Rechazo: sección 8.1
- Tiempo de Permanencia: sección 8.2
- Ocupación Promedio: sección 8.3

8.5. Conclusión

A partir del análisis de los intervalos de confianza al 95 %, se observa que la **Tasa de Rechazo** presenta alta variabilidad e incertidumbre, aunque su valor medio (18 %) sugiere que una proporción significativa de vehículos no logra ingresar al sistema. El **Tiempo de Permanencia** se encuentra entre 217 y 365 segundos, indicando una duración moderada dentro del sistema. Por su parte, la **Ocupación Promedio** oscila entre 14 y 27 vehículos, reflejando una alta utilización del espacio disponible. Estos resultados evidencian un sistema con uso intensivo, posibles cuellos de botella y la necesidad de optimizar su capacidad para mejorar la eficiencia y reducir rechazos.

9. Análisis de Propiedades

9.1. Safety

Esta propiedad debe mostrar que en todo experimento realizado la ocupación máxima nunca supera la capacidad máxima del estacionamiento.

En nuestro sistema, utilizando una capacidad máxima de 30, los experimentos van a cumplir siempre con esta propiedad porque el controlador revisa en las peticiones de entrada al estacionamiento si la cantidad de autos actual en el estacionamiento es menor a 30 o igual a 30.

En caso que la cantidad de autos sea menor a 30, el auto que quiere entrar es permitido y se le avisa a la barrera para abrirse y el auto pasa al estacionamiento.

En caso que la cantidad de autos sea igual a 30, el auto queda demorado como máximo 3 segundos.

Si pasan los 3 segundos y ningún auto se retiró, se le avisa a la barrera que no se abra y el auto se retira luego de 2 segundos. Cabe destacar que en este proceso de rechazar nunca se sumó la cantidad de autos, por lo que sigue siendo 30.

Si dentro de los 3 segundos se encontró que algún auto se retiró, el auto sigue un proceso igual al que alguien que entra como si hubiera cantidad de autos menor a 30. Y lo que pasa con la cantidad de autos es que disminuyó en 1 unidad cuando salió alguien y vuelve a subir en 1 unidad nuevamente con este auto, manteniendo así que la cantidad de autos es igual a 30.

9.2. Liveness 1

Esta propiedad debe mostrar que todo auto que entró al estacionamiento se retiró antes del tiempo final de simulación.

En el sistema, todos los experimentos cumplen con esta propiedad porque el sensor de entrada va a dejar de detectar autos y enviarlos al controlador cuando el tiempo total de apertura del estacionamiento se concluye.

Para esto, se compara el tiempo actual de simulación contra el tiempo total de apertura del estacionamiento. En caso de que esto ocurra, se activa un flag de fin de simulación y dejará bloqueado el sensor indefinidamente, evitando que los autos generados por el generador exponencial pasen a etapas del controlador y las barreras.

Esto siempre será posible si se pone un tiempo de simulación mayor que el tiempo total de apertura del estacionamiento.

Una vez el sensor queda bloqueado por fin de simulación, ahora el controlador, el estacionamiento, la barrera de salida y el sensor de salida son los encargados de vaciar el estacionamiento naturalmente.

9.3. Liveness 2

Esta propiedad se encarga de revisar que cada auto dentro del estacionamiento no exceda el 1 % de su tiempo de estacionado al salir del estacionamiento

Esta propiedad en ningún caso se cumple, ni siquiera en el peor caso de 300 segs, porque se le está pidiendo que cada auto no demore más de 1 % de tiempo en salir. En este caso tiene hasta 3 segundos de tolerancia para salir, pero ya sabemos que las barreras tardan un mínimo de 4 segundos en abrirse, y 4 en cerrarse, además del tiempo asignado por la distribución uniforme en cruzar la barrera, por lo que nunca se puede cumplir esta propiedad.

Se debería aumentar el porcentaje de tolerancia permitido en salir, que mínimo sea mayor a 4 segundos (el tiempo mínimo de la barrera en abrirse) más 4 (el tiempo mínimo de la barrera en cerrarse) más 3 (el tiempo máximo para cruzar la barrera) más 1 (el tiempo que tarda el sensor en detectar el vehículo).

Este nuevo cambio es ligeramente más permisivo, pero tampoco asegura que se cumpla siempre por culpa de las colas de salida.

Para que esta propiedad pueda cumplirse en algún momento, se podría pedir un porcentaje más razonable, como 10 % o más por ejemplo. También se podría reducir el tiempo de apertura y cerrado de la barrera de salida, o el tiempo de detectar un vehículo en el sensor de salida.

10. Discusión y Sugerencias de Mejora

10.1. Impacto de parámetros en rechazos y ocupación.

Los parámetros que se usen para los distintos atomics afectan al desempeño del sistema sin dudas. Modificar la capacidad del estacionamiento, las distintas velocidades tales como detección en los sensores, apertura o cierre en las barreras serían posibles cambios en un escenario real, y con la conclusión de los distintos experimentos simulados se puede saber qué podría llegar a pasar, que es una de las ideas principales de simular sistemas.

10.2. Posible extensión: múltiples barreras, mayores velocidades o capacidad en el estacionamiento.

Basándonos en el experimento 1, que es la implementación base del sistema simulado, los resultados obtenidos denotan un sistema bastante estable, porque se mantiene una ocupación alta y sin rechazos.

Si pensáramos en una posible extensión sería en añadir más barreras, pero a coste de añadir más plazas al estacionamiento, porque los resultados estadísticos arrojan que la capacidad es muy alta, sin llegar al límite, pero si se añade una barrera más se piensa en que llegan al estacionamiento el doble de autos, por lo que es fundamental que se añadan plazas extra para no producir rechazos en exceso.

Luego, cada experimento refleja una situación en particular, en donde los recursos añadidos o quitados en cada uno de ellos depende puntualmente de la situación que refleje.

Hay una buena discusión al hablar de mejorar las distintas velocidades presentes en el sistema, pensando en la experiencia del usuario en cuanto a las demoras reales que puede tener, que eventualmente recae sobre simular un sistema realista.

En este sentido, uno podría decir que simular velocidades realmente rápidas para sus componentes atraería una mejora en la experiencia del usuario, pero que son físicamente

imposibles de llevar a la realidad, por lo que una simulación de ese estilo es meramente virtual, y el sistema simulado no se puede llevar a cabo en la realidad.

11. Conclusiones

El sistema de simulación implementado demuestra un correcto funcionamiento general, cumpliendo con las propiedades fundamentales de *Safety* y *Liveness 1* en todos los escenarios evaluados. Específicamente:

- **Safety:** En ninguna de las simulaciones la ocupación del estacionamiento superó el valor máximo permitido de 30 vehículos, validando la lógica de control establecida en el modelo.
- **Liveness 1:** Todos los vehículos admitidos en el sistema lograron egresar antes de finalizar la simulación, garantizando la fluidez del sistema bajo las condiciones planteadas.
- **Liveness 2:** Esta propiedad no se cumplió en ningún experimento, dado que los tiempos físicos de detección, apertura y cruce superan el 1 % de tolerancia establecida. Esto señala la necesidad de revisar dicha restricción o adaptar los tiempos operativos si se busca cumplirla en el futuro.

11.1. Recomendaciones para la operación real

A partir del análisis de los experimentos, se destacan las siguientes recomendaciones prácticas para un sistema real:

- **Ajustar la capacidad del estacionamiento** en función de la demanda proyectada. Simulaciones con capacidad reducida evidenciaron una alta tasa de rechazo que afecta negativamente la eficiencia del sistema.
- **Mejorar los tiempos de respuesta de los componentes**, como sensores y barreras, siempre que sea físicamente factible. Esto permitiría incrementar el rendimiento sin comprometer la seguridad operativa.
- **Evitar la saturación prolongada** en escenarios de alta permanencia, mediante políticas que incentiven la rotación de vehículos o con mecanismos de reserva de plazas.
- **Tolerancia más realista para Liveness 2:** Ajustar el umbral de cumplimiento de esta propiedad a un valor factible (por ejemplo, 10 % del tiempo de permanencia) permitiría verificar su cumplimiento sin forzar restricciones inviables en contextos físicos reales.

11.2. Líneas futuras de investigación

Se sugieren a continuación posibles ampliaciones del trabajo realizado:

- **Simulación con múltiples barreras de ingreso o egreso**, evaluando el impacto en la tasa de rechazo y en la ocupación bajo diferentes esquemas de coordinación y prioridad.

- **Modelado de fallas en sensores o barreras**, para incorporar análisis de confiabilidad y robustez operativa ante situaciones no ideales.
- **Evaluación de estrategias dinámicas**, como admisión condicional basada en predicciones de egresos, o modulación de tarifas según ocupación, permitiendo un control adaptativo más sofisticado.
- **Validación empírica con datos reales** de estacionamientos automatizados, para calibrar los parámetros del modelo y comparar los resultados de la simulación con escenarios reales.
- **Integración con sistemas IoT y tecnologías de Smart City**, con el fin de estudiar su viabilidad en un entorno urbano inteligente.

11.3. Reflexión final

La simulación de este tipo de sistemas ofrece un entorno controlado donde analizar y comparar distintas configuraciones operativas sin incurrir en costos reales. Los resultados obtenidos permiten extraer conclusiones relevantes tanto para el diseño de sistemas automatizados como para su posterior evaluación y mejora continua. Así, este trabajo sienta una base sólida para futuras aplicaciones y estudios en el ámbito de la simulación de sistemas de acceso vehicular inteligente.

12. Referencias

1. Especificación del proyecto (PowerDEVS)
2. Zeigler, B. P., “Theory of Modeling and Simulation,” 2018.
3. Wainer, G., “Discrete-Event Modeling and Simulation,” 2009.
4. Cristiá, Maximiliano & Hollmann, Diego & Frydman, Claudia. (2018). A multi-target compiler for CML-DEVS. SIMULATION. 95. 003754971876508. 10.1177/0037549718765080.
5. Diego A. Hollmann, Maximiliano Cristiá, Claudia Frydman, CML-DEVS: A specification language for DEVS conceptual models, Simulation Modelling Practice and Theory, Volume 57, 2015, Pages 100-117, ISSN 1569-190X, <https://doi.org/10.1016/j.simpat.2015.06.007>.
6. Hollmann, D. A. (2014). Traducción y validación sistemática y automática de modelos DEVS abstractos (Tesis doctoral). Universidad Nacional de Rosario, Facultad de Ciencias Exactas, Ingeniería y Agrimensura, Rosario, Argentina.